
第10章 ROS2 基础及小车实现

10.1 ROS2 简介及安装

10.1.1 ROS 到 ROS2 的发展历程

1. ROS（ROS1）的诞生与发展

ROS（Robot Operating System）起源于 2007 年斯坦福大学机器人研究实验室的研究项目，随后在 2010 年由 Willow Garage 推动工程化和社区化发展，并逐步成长为机器人领域中影响力极大的开源软件框架。虽然名称中包含“操作系统”，但 ROS 实际上并不替代 Linux 等底层操作系统，而是在其之上提供一套面向机器人应用的软件基础设施，包括通信机制、软件组织方式以及开发调试工具链。

ROS1 建立了一种以节点为基本单元的分布式软件模型。在这种模型中，不同功能模块以独立节点进程的形式运行，例如传感器驱动、感知算法、定位与控制模块等，各节点之间通过话题机制进行数据发布与订阅，通过服务机制完成请求—响应式调用，从而构成松耦合的软件系统结构。系统功能通常以功能包为单位进行组织，方便复用与共享。同时，ROS1 提供了较为完善的开发工具体系，例如 `rviz` 用于三维可视化调试，`rosbag` 用于数据记录与回放，这些工具与大量开源功能包共同构建了活跃而完整的生态环境，使 ROS1 在科研与教学场景中得到广泛应用。

2. ROS1 的局限性

随着机器人系统逐渐从实验研究走向复杂工程应用，ROS1 的早期架构设计开始暴露出一些约束条件。ROS1 的节点发现与通信关系依赖中心化的 Master 机制进行管理，一旦 Master 节点异常，将对整个系统的通信拓扑产生影响，这种单点依赖在大规模系统中存在可靠性风险。在通信层面，ROS1 主要基于 TCPROS 与 UDPROS 机制实现数据传输，虽然具备良好的通用性，但缺乏严格的实时性保障机制，在高频闭环控制和工业实时场景中难以满足需求。

在多机器人系统与跨网络部署环境中，ROS1 的配置复杂度也会明显上升，不同网络域之间的通信需要额外桥接与参数配置，系统扩展成本较高。同时，ROS1 在最初设计时并未将安全机制作为核心目标，因此缺乏内置的身份认证与加密通信能力，在工业网络或开放网络环境中存在安全隐患。从平台适配角度看，ROS1 主要围绕 Linux 环境构建，对多操作系统与嵌入式平台的支持能力有限。这些因素共同限制了 ROS1 在产业级应用中的进一步推广。

3. ROS2 的提出与发展

为解决 ROS1 在可靠性、实时性、安全性与跨平台能力方面的不足，开源机器人基金会 OSRF 自 2014 年起启动了 ROS2 的重新架构设计工作。ROS2 并不是在 ROS1 基础上的简单升级，而是一次从通信机制到系统架构的整体重构，其目标是构建面向长期工程应用与产业部署的新一代机器人软件平台。

ROS2 在底层通信机制上采用了工业领域广泛应用的数据分发服务 DDS 作为中间件基础，实现了去中心化的节点发现与数据分发机制，从架构层面消除了中心节点单点故障问题。DDS 提供的服务质量策略允许开发者根据实时性与可靠性需求对通信行为进行参数化配置，使系统能够适配不同等级的控制与数据流场景。在系统能力方面，ROS2 明确支持实时操作系统环境与实时内核配置，更适合高频控制任务。同时，ROS2 官方支持 Linux、Windows 与 macOS 等多种操作系统，并能够向嵌入式平台扩展。在安全方面，ROS2 引入标准化的认证、加密与访问控制机制，使系统在工程部署中具备更好的安全基础。结合组件化设计与节点生命周期管理机制，ROS2 在系统可维护性与可扩展性方面也较 ROS1 有明显提升。

10.1.2在 Ubuntu 中部署 ROS2 开发环境

1. 手动安装 ROS2

1) 下载 ROS2 发行版

(1) 查看 Ubuntu 版本，据此选择 ros 版本

```
lsb_release -a
```

```
test@test-vm:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 22.04.4 LTS
Release:        22.04
Codename:       jammy
```

ROS2版本	生命周期	Ubuntu版本	语言依赖
Foxy Fitzroy	2020.5-2023.5	Ubuntu Focal (20.04)	C++ >=14、Python >= 3.7
Galactic Geochelone	2021.5-2022.11	Ubuntu Focal (20.04)	C++ >=17、Python >=3.6
Humble Hawksbill	2022.5-2027.5	Ubuntu Jammy (22.04)、Ubuntu Focal (20.04)(只能源码安装)	C++ >=17 、Python >=3.6
Iron Irwini	2023.5-2024.11	Ubuntu Jammy (22.04)	C++ >=17 、Python >= 3.8
Jazzy Jalisco	2024.5-2029.5	Ubuntu Noble (24.04)	C++ >=17 、Python >= 3.8

本书以 Ubuntu22.04 ROS2 Humble 版为例

(2)设置 locale

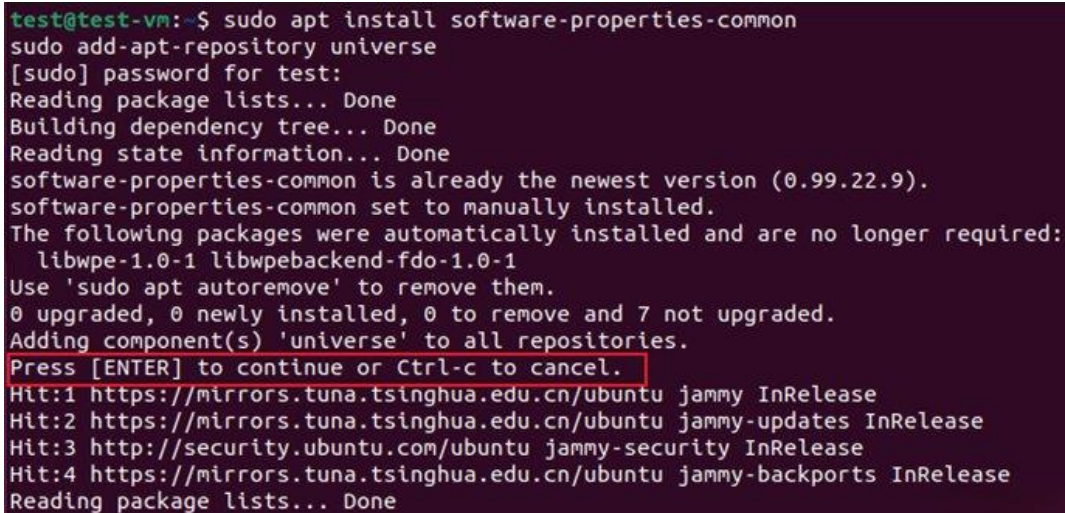
确保本地语言支持 UTF-8

```
sudo apt update && sudo apt install locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8
```

2)设置源

(1)启用 Universe 源

```
sudo apt install software-properties-common
sudo add-apt-repository universe
```



```
test@test-vm:~$ sudo apt install software-properties-common
sudo add-apt-repository universe
[sudo] password for test:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
software-properties-common is already the newest version (0.99.22.9).
software-properties-common set to manually installed.
The following packages were automatically installed and are no longer required:
  libwpe-1.0-1 libwpebackend-fdo-1.0-1
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 7 not upgraded.
Adding component(s) 'universe' to all repositories.
Press [ENTER] to continue or Ctrl-c to cancel.
Hit:1 https://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy InRelease
Hit:2 https://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-updates InRelease
Hit:3 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:4 https://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-backports InRelease
Reading package lists... Done
```

(2)添加 ROS 2 GPG key

```
sudo apt update && sudo apt install curl gnupg2 -y
sudo curl -sSL https://gitee.com/tyx6/rosdistro/raw/master/ros.key -o
/usr/share/keyrings/ros-archive-keyring.gpg
```

(3)为软件源添加 ROS2 repository

可选择的软件源包括官方源、清华源、中科大源等

(4)官方源

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-
archive-keyring.gpg] http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo
$UBUNTU_CODENAME) main" | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
```

(5)清华源

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-
archive-keyring.gpg] https://mirrors.tuna.tsinghua.edu.cn/ros2/ubuntu $(. /etc/os-release
&& echo $UBUNTU_CODENAME) main" | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
```

(6)安装

本书以 Desktop 版为例，包含: ROS, RViz, demos, tutorials。

base 版仅包含: Communication libraries, message packages, command line tools。。不包含 GUI tools。

```
sudo apt update && sudo apt upgrade
sudo apt install ros-humble-desktop
```

(7)配置 ~/.bashrc 环境

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
source ~/.bashrc #使环境生效
```

3)配置 rosdep

在使用许多 ROS 工具之前，需要初始化 rosdep，有些功能包源码编译需要 rosdep 来安装这些系统依赖项，不配置也不影响 ROS 使用，所以后面需要时再来配置也可以。

这个 rosdep 请求的是国外的服务器，所以会被墙。有些是通过代理的方式，但这个不稳定，因为时间久了要么代理跑路了，要么被墙了。它需要的文件都官方都放在 github 上的，那么我们可以改 url 地址即可。

(1)安装依赖

```
sudo apt install python3-rosdep
```

(2)自动修改

此脚本在 rosdistro 目录下，是为了方便编写的，如果脚本执行失败可以手动修改

```
wget https://gitee.com/tyx6/mytools/raw/main/ros/Mrosdep.py
sudo python3 Mrosdep.py
```

```
test@test-vm:~/src/rosdistro$ sudo python3 Mrosdep.py
欢迎使用脚本，本脚本适用于ROS1、ROS2.
Test: ROS1 Melodic、ROS2 Noetic、ROS2 Humble
The ROS version is: humble
./rosdep/sources.list.d/20-default.list is ok
/usr/lib/python3/dist-packages/rosdep2/sources_list.py is ok
/usr/lib/python3/dist-packages/rosdistro/__init__.py is ok
/usr/lib/python3/dist-packages/rosdep2/gbpdistro_support.py is ok
/usr/lib/python3/dist-packages/rosdep2/rep3.py is ok
修改成功！请执行下面命令：
sudo rosdep init
rosdep update
```

(3)手动修改

修改 4 个文件，都是将地址 <https://raw.githubusercontent.com/...> 改为

<https://gitee.com/tyx6/rosdistro/raw/master/>

```
sudo gedit /usr/lib/python3/dist-packages/rosdep2/sources_list.py
```

```
59 from .loader import RosdepLoader
60 from .rosdistrohelper import get_index, get_index_url
61
62 # default file to download with 'init' command in order to bootstrap
63 # rosdep
64 DEFAULT_SOURCES_LIST_URL = 'https://gitee.com/tyx6/mytools/raw/main/ros/20-default.list'
65
66 # seconds to wait before aborting download of rosdep data
67 DOWNLOAD_TIMEOUT = 15.0
68
```

```
sudo gedit /usr/lib/python3/dist-packages/rosdistro/__init__.py
```

```
64 __version__ = '0.9.1'
65
66 # index information
67
68 DEFAULT_INDEX_URL = 'https://gitee.com/tyx6/rosdistro/raw/master/index-v4.yaml'
69
70
```

```
sudo gedit /usr/lib/python3/dist-packages/rosdep2/gbpdistro_support.py
```

```
30 except NameError:
31     basestring = unicode = str
32
33 # location of an example gbpdistro file for reference and testing
34 FUERTE_GBPDISTRO_URL = 'https://gitee.com/tyx6/rosdistro/raw/master/releases/fuerte.yaml'
35
36 # seconds to wait before aborting download of gbpdistro data
```

```
sudo gedit /usr/lib/python3/dist-packages/rosdep2/rep3.py
```

```
33 from .url_utils import urlopen_gzip
34
35 # location of targets file for processing gbpdistro files
36 REP3_TARGETS_URL = 'https://gitee.com/tyx6/rosdistro/raw/master/releases/targets.yaml'
37
38 # seconds to wait before aborting download of gbpdistro data
```

```
sudo rosdep init
```

```
rosdep update
```

```
test@test-wm:~/src/rosdistro$ rosdep update
reading in sources list data from /etc/ros/rosdep/sources.list.d
Hit file:///home/test/src/rosdistro/rosdep/osx-homebrew.yaml
Hit file:///home/test/src/rosdistro/rosdep/base.yaml
Hit file:///home/test/src/rosdistro/rosdep/python.yaml
Hit file:///home/test/src/rosdistro/rosdep/ruby.yaml
Hit file:///home/test/src/rosdistro/releases/fuerte.yaml
Query rosdistro index files:///home/test/src/rosdistro/index-v4.yaml
Skip end-of-life distro "ardent"
Skip end-of-life distro "bouncy"
Skip end-of-life distro "crystal"
Skip end-of-life distro "dashing"
Skip end-of-life distro "eloquent"
Skip end-of-life distro "foxy"
Skip end-of-life distro "galactic"
Skip end-of-life distro "groovy"
Add distro "humble"
Skip end-of-life distro "hydro"
Skip end-of-life distro "indigo"
Add distro "iron"
Skip end-of-life distro "jade"
Add distro "jazzy"
Skip end-of-life distro "kinetic"
Skip end-of-life distro "lunar"
Skip end-of-life distro "melodic"
Add distro "noetic"
Add distro "rolling"
updated cache in /home/test/.ros/rosdep/sources.cache
```

(4)测试

在第一个终端中输入

```
ros2 run demo_nodes_cpp talker
```

添加第二个终端，输入


```
ros2 run demo_nodes_py listener
```

```
test@test-vm: ~  
test@test-vm:~$ ros2 run demo_nodes_cpp talker  
[INFO] [1720603243.944535140] [talker]: Publishing: 'Hello World: 1'  
[INFO] [1720603244.944461139] [talker]: Publishing: 'Hello World: 2'  
[INFO] [1720603245.944508754] [talker]: Publishing: 'Hello World: 3'  
[INFO] [1720603246.943889085] [talker]: Publishing: 'Hello World: 4'  
[INFO] [1720603247.943928333] [talker]: Publishing: 'Hello World: 5'  
[INFO] [1720603248.943948436] [talker]: Publishing: 'Hello World: 6'  
[INFO] [1720603249.944133001] [talker]: Publishing: 'Hello World: 7'  
  
test@test-vm:~/Desktop$ cd  
test@test-vm:~$ ros2 run demo_nodes_py listener  
[INFO] [1720603245.954989628] [listener]: I heard: [Hello World: 3]  
[INFO] [1720603246.944610234] [listener]: I heard: [Hello World: 4]  
[INFO] [1720603247.944653044] [listener]: I heard: [Hello World: 5]  
[INFO] [1720603248.944641096] [listener]: I heard: [Hello World: 6]  
[INFO] [1720603249.944810780] [listener]: I heard: [Hello World: 7]
```

2. 一键安装 ROS2

使用鱼香 ROS 开发的安装工具，依照终端提示完成安装

```
wget http://fishros.com/install -O fishros && . Fishros
```

在终端看到多个安装选项，依据需要进行 ROS2 安装及 rosdep 配置

```
RUN Choose Task:[请输入括号内的数字]  
---众多工具，等君来用---  
ROS相关:  
[1]:一键安装(推荐):ROS(支持ROS/ROS2,树莓派Jetson)  
[3]:一键安装:rosdep(小鱼的rosdep,又快又好)  
[4]:一键配置:ROS环境(快速更新ROS环境设置,自动生成环境选择)  
[9]:一键安装:Cartographer(18 20测试通过,16未测, updateTime 20240125)  
[11]:一键安装:ROS Docker版(支持所有版本ROS/ROS2)  
[16]:一键安装:系统自带ROS (!!警告!! 仅供特殊情况下使用)  
  
常用软件:  
[2]:一键安装:github桌面版(小鱼常用的github客户端)  
[6]:一键安装:NodeJS环境  
[7]:一键安装:VsCode开发工具  
[8]:一键安装:Docker  
[10]:一键安装:微信(可以在Linux上使用的微信)  
[12]:一键安装:PlatformIO MicroROS开发环境(支持Fishbot)  
[14]:一键安装:科学上网代理工具  
[15]:一键安装:QQ for Linux  
  
配置工具:  
[5]:一键配置:系统源(更换系统源,支持全版本Ubuntu系统)  
[13]:一键配置:python国内源  
[17]:一键配置:Docker代理(支持VPN+代理服务两种模式)  
[0]:quit
```

10.2 ROS2 原理和组件工具

10.2.1 ROS2 系统架构

1. 操作系统层

ROS2 通常运行在 Linux、Windows 或 macOS 等操作系统之上。在嵌入式或工业场景中，还可以运行在实时操作系统（RTOS）上，为机器人应用提供基础的计算和资源管理能力。

2. 分布式网络（DDS）实现层

ROS2 的通信基于 DDS（Data Distribution Service）。DDS 提供了去中心化的发布/订阅机制，使得多个节点能够在无中心管理的情况下实现高效、可靠的数据传输。不同 DDS 实现（如 FastDDS、CycloneDDS、RTI Connexx 等）可以作为插件被使用。

3. DDS 接口层

为了让上层开发者无需直接操作 DDS，ROS2 在 DDS 之上提供了一层抽象接口（RMW，ROS Middleware Interface）。RMW 屏蔽了底层 DDS 的差异，保证了开发者编写的 ROS2 节点可以在不同 DDS 实现之间无缝切换。

4. ROS2 客户端层

这一层提供了具体的编程接口（如 rclcpp、rclpy），使得开发者可以使用 C++ 或 Python 编写 ROS2 节点。同时还包括参数服务器、生命周期管理等核心功能。

5. 应用层

应用层是开发者直接接触的部分，包括机器人控制、导航、感知、SLAM 等各种功能包。应用层利用下层的通信和工具支撑，实现具体的机器人应用。

10.2.2 ROS2 组件工具

1. 坐标变换工具

1) TF 变换介绍

在 ROS2 中，**TF (Transform)** 是一个用于管理和维护坐标系之间空间关系的工具包。机器人系统通常包含多个坐标系，例如 **地图坐标系 (map)**、**机器人本体坐标系 (base_link)**、**传感器坐标系 (camera、lidar)** 等。为了让系统能够正确理解各个坐标之间的位置和姿态关系，ROS2 提供了 **tf2 库** 来发布、广播和查询这些坐标变换。

TF 的核心作用在于：

统一坐标系：保证不同传感器、机器人部件的数据能在同一个空间下进行对齐和融合。

时间同步：支持在不同时刻查询任意两个坐标系之间的关系。

方便开发：开发者只需关心逻辑层面的功能，复杂的坐标换算由 TF 自动完成。

借助 TF，机器人可以实现诸如传感器数据融合、路径规划、导航定位等功能，是 ROS2 中的**基础模块之一**。

本节以 TurtleSim 为例介绍 TF 工具。

配置相关环境

```
sudo apt-get install ros-humble-turtle-tf2-py ros-humble-tf2-tools ros-humble-tf-
transformations

sudo apt install python3-pip

pip3 install transforms3d
```

(1)静态 TF 广播

(2)通过命令行静态 TF 广播

```
ros2 run tf2_ros static_transform_publisher
#后面有两个必要节点和九个非必要节点
```

frame_id: 父坐标框架的名称

child_frame_id: 子坐标框架的名称

x: 变换的 x 坐标 (单位: 米)

y: 变换的 y 坐标 (单位: 米)

z: 变换的 z 坐标 (单位: 米)

yaw: 绕 z 轴的旋转 (单位: 弧度)

pitch: 绕 y 轴的旋转 (单位: 弧度)

roll: 绕 x 轴的旋转 (单位: 弧度)

```
The translation offsets are in meters.

The rotation may be provided with roll, pitch, yaw euler angles in radians, or as a quaternion.

required arguments:
  --frame-id FRAME_ID parent frame
  --child-frame-id CHILD_FRAME_ID child frame id

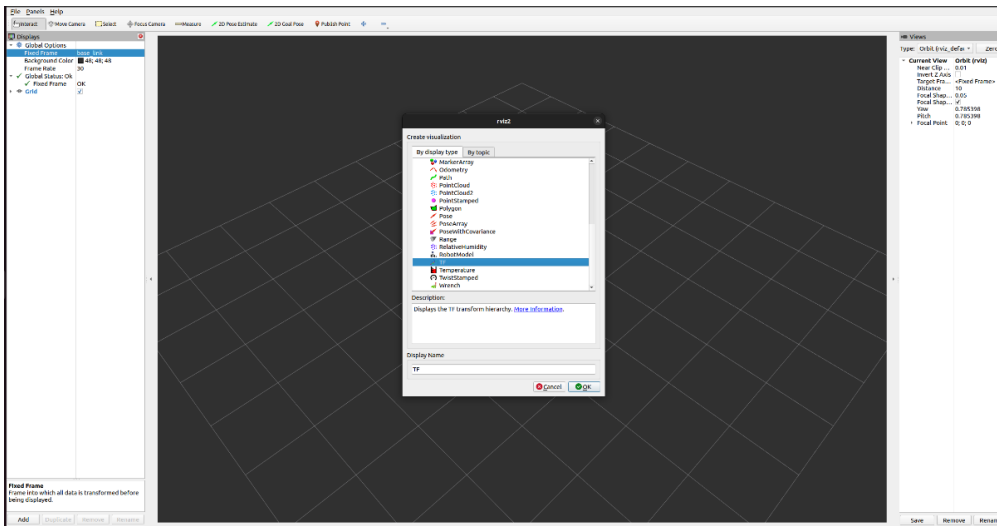
optional arguments:
  --x X           x component of translation
  --y Y           y component of translation
  --z Z           z component of translation
  --qx QX         x component of quaternion rotation
  --qy QY         y component of quaternion rotation
  --qz QZ         z component of quaternion rotation
  --qw QW         w component of quaternion rotation
  --roll ROLL     roll component Euler rotation
  --pitch PITCH   pitch component Euler rotation
  --yaw YAW       yaw component Euler rotation
```

示例代码:

```
ros2 run tf2_ros static_transform_publisher --frame-id base_link --child-frame-id camera --x -0.5 --z 0.4
```

运行完这行代码可以开启新终端启动 rviz2 进行查看

进入界面后将坐标系改为 base_link 坐标系, 在左下角点击 add、添加 tf, 即可看到设置的两个坐标系, 在左边选择栏中的 tf 可以选择是否显示名字和消失事件



(3)通过 Python 进行静态 TF 广播

```
# 配置依赖库
```

```
from tf2_ros import TransformBroadcaster #用于发布数据
```

```
import sys #用于获取参数
```

```

from rclpy.logging import get_logger          #用于打印
from geometry_msgs.msg import TransformStamped #发布数据类型
import tf_transformations                     #四元数转化
#包含头文件
import rclpy
from rclpy.node import Node
from tf2_ros import TransformBroadcaster

import sys
from rclpy.logging import get_logger
from geometry_msgs.msg import TransformStamped
import tf_transformations

# 自定义节点
class TFStationBroadcaster(Node):
    def __init__(self,argv ):
        super().__init__("tf_station_broadcaster_node")
        self.get_logger().info("发布成功")
        self.broadcaster = TransformBroadcaster(self)
        self.pub_static_tf(argv)

    def pub_static_tf(self,argv):

        tf = TransformStamped()

        tf.header.frame_id = argv[7]
        tf.header.stamp = self.get_clock().now().to_msg()
        tf.child_frame_id = argv[8]

        tf.transform.translation.x = float(argv[1])
        tf.transform.translation.y = float(argv[2])
        tf.transform.translation.z = float(argv[3])

        qtn =
tf_transformations.quaternion_from_euler(float(argv[4]),float(argv[5]),float(argv[6]))

```

```

        tf.transform.rotation.x = qtn[0]
        tf.transform.rotation.y = qtn[1]
        tf.transform.rotation.z = qtn[2]
        tf.transform.rotation.w = qtn[3]
        self.broadcaster.sendTransform(tf)

def main():

    if(len(sys.argv) != 9):
        get_logger("rclpy").error("传入的参数不合法")
        return 1
    # 初始化客户端
    rclpy.init()
    # 调用 spin 传入节点对象
    rclpy.spin(TFStationBroadcaster(sys.argv))
    # 释放资源
    rclpy.shutdown()

if __name__=='__main__':
    main()

```

(4)运行

```
ros2 run cpp03_tf_broadcaster demo01_static_tf_broadcaster 0.2 0.0 0.4 0 0 0 base_link laser
```

(5)动态 TF 广播

动态广播需要获取目标点的位置信息，然后通过类似静态广播的广播发布方发布出去（动态广播没有命令行执行）

编写动态[坐标变换](#)程序，启动 `turtlesim_node` 以及 `turtle_teleop_key` 后，该程序可以发布乌龟坐标系到窗口坐标系的坐标变换，并且键盘控制乌龟运动时，乌龟坐标系与窗口坐标系的相对关系也会实时更新。

(6)通过 Python 实现动态 TF 广播

为了获取乌龟的坐标信息，需要开启乌龟话题的接收方获取 `pose` 信息，再将 `pose` 信息传递给广播信息中发布出去，由于 `pose` 信息中的的翻转、俯仰和偏航角是由三个参数给出，但广播中需要将其转换为四元数，这里就需要导入 `#include "tf2/LinearMath/Quaternion.h"`来进行转换。

```

#包含头文件

import rclpy
from rclpy.node import Node

from rclpy.logging import get_logger
from tf2_ros.static_transform_broadcaster import StaticTransformBroadcaster
from geometry_msgs.msg import TransformStamped
import tf_transformations
from turtlesim.msg import Pose

# 自定义节点
class TF_Dynamic_Broadcaster(Node):
    def __init__(self):
        super().__init__("TF_Dynamic_Broadcaster_node")
        self.get_logger().info("启动成功")
        self.broadcaster_ = StaticTransformBroadcaster(self)
        self.pose_pub_ = self.create_subscription(Pose, "/turtle1/pose", self.do_pose, 10)

    def do_pose(self, pose: Pose):
        transform = TransformStamped()
        transform.child_frame_id = "turtle1"
        transform.header.frame_id = "world"
        transform.header.stamp = self.get_clock().now().to_msg()
        transform.transform.translation.x = pose.x
        transform.transform.translation.y = pose.y
        transform.transform.translation.z = 0.0
        qtn = tf_transformations.quaternion_from_euler(0.0, 0.0, pose.theta)
        transform.transform.rotation.x = qtn[0]
        transform.transform.rotation.y = qtn[1]
        transform.transform.rotation.z = qtn[2]
        transform.transform.rotation.w = qtn[3]
        self.broadcaster_.sendTransform(transform)

def main():
    # 初始化客户端

```

```

rclpy.init()

# 调用 spin 传入节点对象

rclpy.spin(TF_Dynamic_Broadcaster())

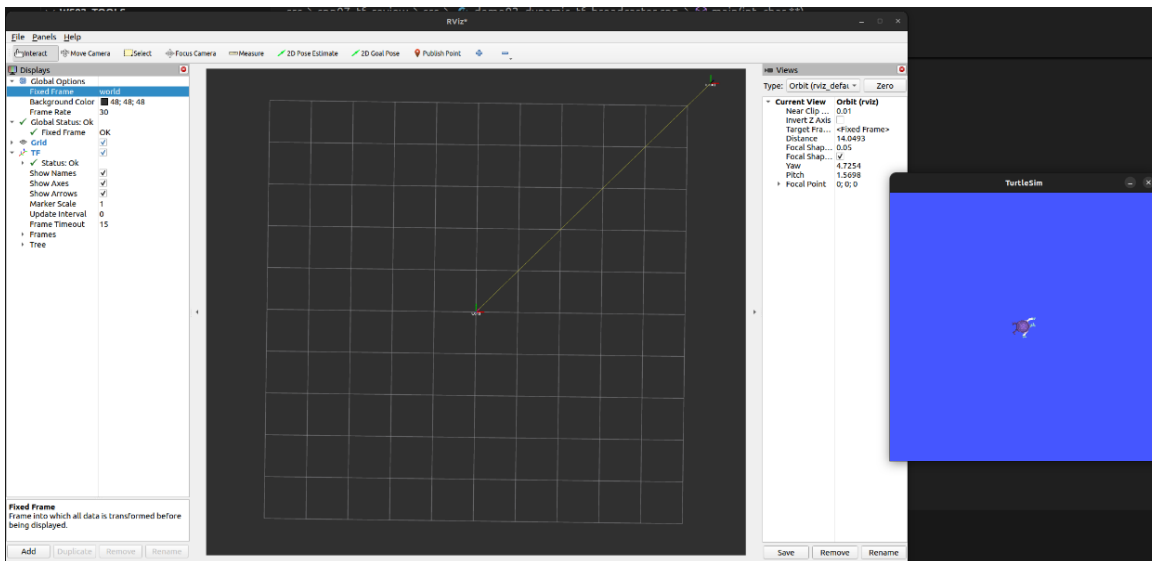
# 释放资源

rclpy.shutdown()

if __name__=='__main__':

    main()

```



(7)监听 TF 变换

前面都是发布广播位置信息，这里是对广播信息进行获取与解析

假设发布广播 A 对 B 的坐标为(1,2),再发布广播 C 对 B 的坐标为 (3,3)，那 A 对 C 的坐标为多少，这个事情就是通过坐标变换监听来获得的。

坐标变换监听是多对一实现的，不同于之前的话题通信，这就需要接收方处理源源不断的不同发布者的信息，为了高效性和稳定性，这里引入了缓存 buffer，当接收方接收到信息后会将信息传给缓存池，当不同信息进入缓存池后就会形成坐标树，我们通过坐标树就可以知道不同坐标之间的关系。

通过 Python 实现监听:

先通过命令行创建三个参照物 base_link,camera 和 laser，求 laser 相对于 camera 的坐标

```

ros2 run tf2_ros static_transform_publisher --frame-id base_link --child-frame-id
laser --x 0.4 --z 0.2

```

```

ros2 run tf2_ros static_transform_publisher --frame-id base_link --child-frame-id
camera --x 0.5 --z 0.4

```

由于信息不一定会同时接收，所以不是每次都可以成功处理两个数据，这个错误是正常的，不需要系统去停止处理错误，所以我们这里使用了 `try`，当没有接收到时发送异常，但不影响程序进行。

最后结果

```
#包含头文件
import rclpy
from rclpy.node import Node

from tf2_ros import TransformListener
from tf2_ros import Buffer
from rclpy.time import Time
# 自定义节点
class My_node(Node):
    def __init__(self):
        super().__init__("My_node_node")
        self.get_logger().info("启动成功")
        self.buffer_ = Buffer()
        self.listener_ = TransformListener(self.buffer_,self)
        self.timer_ = self.create_timer(1.0,self.time_callback)
    def time_callback(self):
        if self.buffer_.can_transform("camera","laser",Time()):
            ts = self.buffer_.lookup_transform("camera","laser",Time())
            self.get_logger().info("-----转换成功-----")
            self.get_logger().info(
                "转换后的父坐标下为: %s , 子坐标系为%s , 相对坐标为(%.2f,%.2f,%.2f)"
                %(ts.header.frame_id,
                  ts.child_frame_id,
                  ts.transform.translation.x,
                  ts.transform.translation.y,
                  ts.transform.translation.z
                )
            )
        else:
            self.get_logger().info("转换失败")
```

```
def main():
    # 初始化客户端
    rclpy.init()

    # 调用 spin 传入节点对象
    rclpy.spin(My_node())

    # 释放资源
    rclpy.shutdown()

if __name__=='__main__':
    main()
```

```
Summary: 1 package finished [19.1s]
user@user-virtual-machine:~/ws02_tools$ ros2 run cpp08 tf_listener demo01_tf_listener
[INFO] [1737560962.266997962] [TF_Listener_node]: 启动成功
[INFO] [1737560963.273571188] [TF_Listener_node]: -----转换完成-----
[INFO] [1737560963.274248547] [TF_Listener_node]: 转换后的子坐标下为: camera, 副坐标系为laser, 相对坐标为 (-0.10,0
00,-0.20)
[INFO] [1737560964.273078194] [TF_Listener_node]: -----转换完成-----
[INFO] [1737560964.273190769] [TF_Listener_node]: 转换后的子坐标下为: camera, 副坐标系为laser, 相对坐标为 (-0.10,0
00,-0.20)
[INFO] [1737560965.273396568] [TF_Listener_node]: -----转换完成-----
[INFO] [1737560965.273524990] [TF_Listener_node]: 转换后的子坐标下为: camera, 副坐标系为laser, 相对坐标为 (-0.10,0
```

2. 常用可视化工具：rqt 和 Rviz

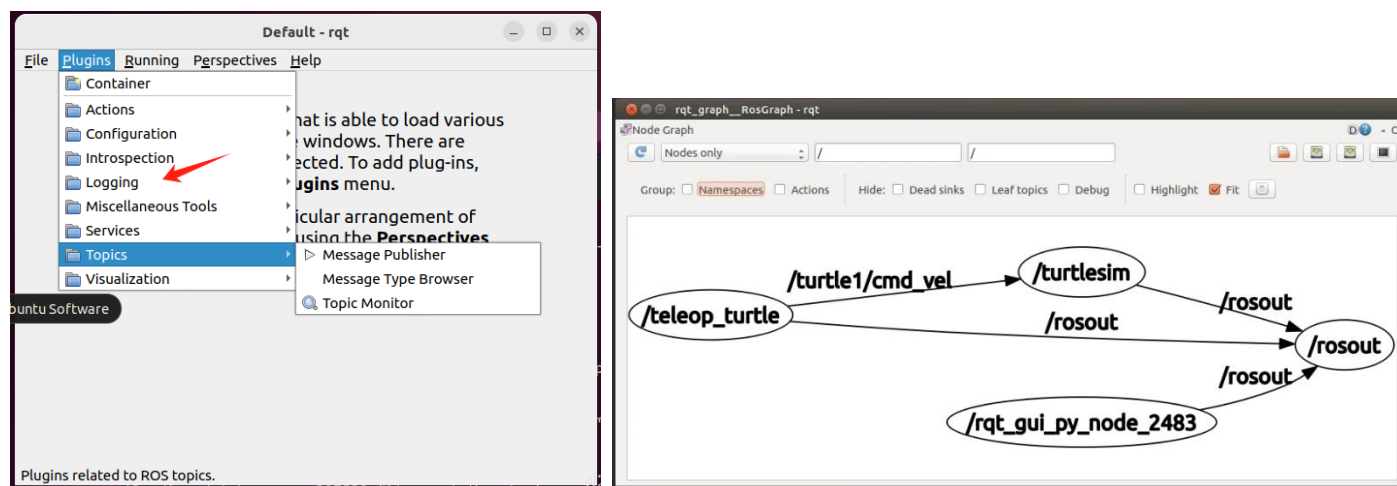
rqt 是 ROS2 提供的一套基于 Qt 的图形化工具框架，它本身不是单一功能软件，而是一个可加载多种插件的可视化工具平台。通过不同插件的组合，用户可以在一个窗口中完成节点管理、话题监视、参数查看、消息发布与调试等多种操作。在系统调试阶段，rqt 常被用作运行状态观察与交互调试入口。

启动 rqt 后，界面最核心的特点是“插件化工作区”。用户可以通过菜单加载不同功能插件，例如节点图查看、话题列表、参数配置界面等，各插件以面板形式加入工作区并支持自由排布。这种设计使 rqt 非常适合在调试过程中按需组合工具视图，而不需要频繁切换命令行窗口。典型使用方式是先加载节点关系图插件观察系统通信结构，再加载话题监视插件查看数据流状态，从而快速判断系统是否按预期运行。

rqt 最常用的功能包括节点关系图可视化、话题消息查看以及简单消息发布调试。节点关系图插件可以用图结构方式展示节点与话题之间的连接关系，适合用于理解系统整体通信拓扑结构；话题查看插件可以实时显示消息内容，便于验证传感器或算法输出是否正常；消息发布插件则可以人工构造测试消息，用于联调尚未完成的模块。图 10-x 展示了 rqt 工作区加载多个插件后的典型界面布局。

在终端输入 rqt 以启动

```
rqt
```

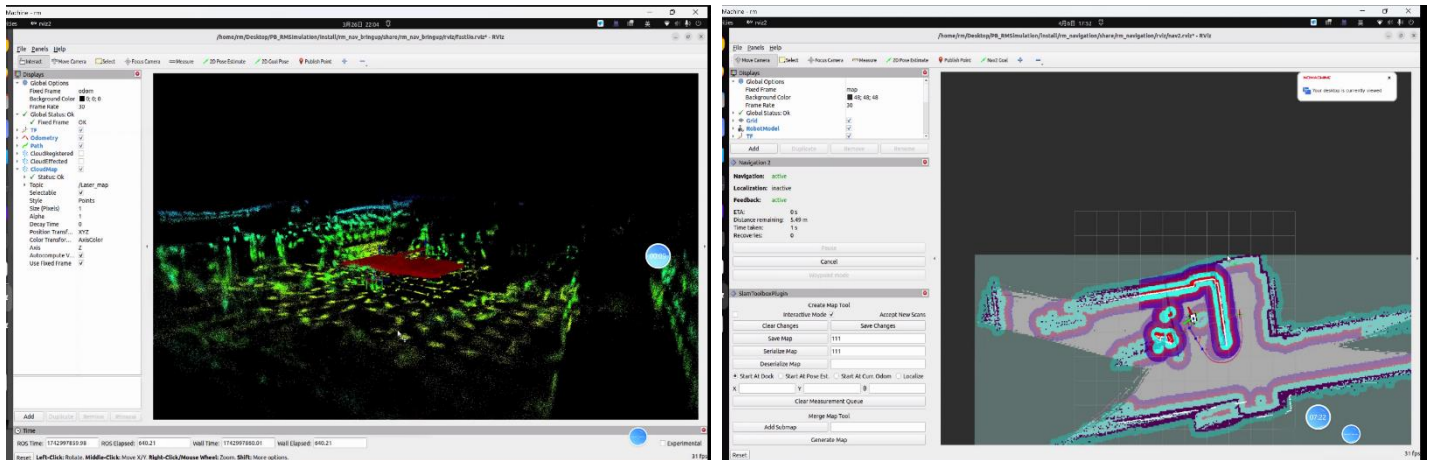



rviz 是 ROS2 中最重要的三维可视化工具之一，主要用于机器人状态、传感器数据与环境模型的可视化展示。在导航、建图、定位与感知算法开发过程中，rviz 往往是最核心的调试界面。通过 rviz，开发者可以将原本抽象的坐标变换、点云数据、地图结果和路径规划结果以三维形式直观呈现，从而快速判断算法行为是否正确。

rviz 的工作方式基于“显示项配置”模型。用户在界面中添加不同类型的显示组件，每一种组件对应一种数据类型或可视化对象，例如坐标系、机器人模型、激光点云、栅格地图或路径轨迹等。每个显示项都需要指定对应的话题名称与坐标系参数，rviz 会自动订阅数据并实时刷新画面。这种机制使 rviz 可以灵活适配不同机器人系统的数据结构，而无需修改程序代码。

首次使用 rviz 时，通常需要先设置全局参考坐标系，然后逐步添加常见显示项，例如坐标轴、机器人模型和传感器数据话题。对于移动机器人系统，还可以叠加地图、代价地图和规划路径等显示层，从而在一个界面中同时观察环境与运动结果。图 10-x 展示了 rviz 中同时加载机器人模型、激光雷达点云与坐标系显示后的典型效果。

在实际工程流程中, `rviz` 常与 `rosbag` 数据回放或在线传感器数据配合使用, 用于算法调参与结果验证。相比命令行数据输出, `rviz` 提供的空间可视化结果更直观。



3. 数据记录工具: `ros2 bag`

`ros2 bag` 是一个命令行工具用于记录话题上发布的消息。他可以保存任意数量话题上的消息内容。我们可以通过数据回放实现实验场景的重现。同时这也是和他人分享数据的好方法。

本节 `ros2bag` 的讲解以 `TurtleSim` 为例

1) 安装 `ros2bag`

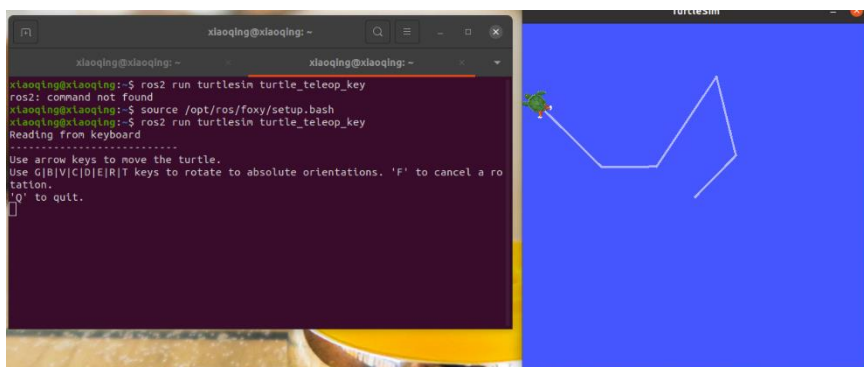
```
sudo apt install ros-eloquent-ros2bag ros-eloquent-roslaunch
```

2) 准备节点和话题

我们将使用 `rosbag` 记录海龟教程中的键盘输入, 首先启动 `TurtleSim` 相应节点

```
ros2 run turtlesim turtlesim_node
```

```
ros2 run turtlesim turtle_teleop_key
```



我们新建一个文件夹用于存放 `rosbag` 文件

```
mkdir bag_files
```

`ros2bag` 只能记录话题的消息, 我们首先查看当前的话题列表

```
ros2 topic list
```

目前所有话题列表如下：

```
/parameter_events
/rosout
/turtle1/cmd_vel
/turtle1/color_sensor
/turtle1/pose
```

比如，/turtle1/cmd_vel 话题就是键盘控制的指令发送的话题，通过下面命令查看话题的消息内容。

```
ros2 topic echo /turtle1/cmd_vel
```

在键盘控制终端没有操作的时候话题上是不会有消息的，按一下方向键消息才会发送到话题上。

3)Ros2bag record

使用 `ros2 bag record <topic_name>`命令可以将制定话题保存到 rosbag 文件

首先切换进入刚才创建的保存文件目录

```
cd bag_files
```

选择记录/turtle1/cmd_vel 话题上的消息

```
ros2 bag record /turtle1/cmd_vel
```

在终端将会看到如下日志，表明开始记录消息

```
[INFO] [rosbag2_storage]: Opened database 'rosbag2_2020_02_20-
11_44_13/rosbag2_2020_02_20-11_44_13_0.db3' for READ_WRITE.
[INFO] [rosbag2_transport]: Listening for topics...
[INFO] [rosbag2_transport]: Subscribed to topic '/turtle1/cmd_vel'
[INFO] [rosbag2_transport]: All requested topics are subscribed. Stopping discovery...
```

现在开始用键盘控制海龟运动，行进一段轨迹之后，在 `ros2 bag` 终端按下 `Ctrl+C` 终止 `ros2 bag` 记录，然后文件会被保存在当前目录下的按照 `rosbag2_year_month_day-hour_minute_second` 方式命令的一个目录下面。

`ros2bag` 也可以记录多个话题甚至全部话题。另外使用 `-o` 选项手动设置输出的文件名字，`-a` 选项记录所有话题。

```
ros2 bag record -o subset /turtle1/cmd_vel /turtle1/pose
```

4)Ros2bag info

通过如下命令查看 rosbag 的信息

```
ros2 bag info <bag_file_name>
```

例如

```
ros2 bag info subset
```

将得到如下输出

```
Files:          subset/subset_0.db3
Bag size:       100.6 KiB
Storage id:     sqlite3
Duration:       12.57s
Start:          Feb 20 2020 11:53:35.524 (1582170815.524)
End             Feb 20 2020 11:53:47.581 (1582170827.581)
Messages:       801
Topic information: Topic: /turtle1/pose | Type: turtlesim/msg/Pose | Count: 755 |
Serialization Format: cdr
                  Topic: /turtle1/cmd_vel | Type: geometry_msgs/msg/Twist | Count: 46 |
Serialization Format: cdr
```

5)ros2 bag play

在回放之前，首先关闭键盘控制节点，但是保留 TurtleSim 主节点，在命令行输入回放 rosbag 命令

```
ros2 bag play subset
```

终端将输出如下消息，海龟将按照之前的键盘指令运动

```
[INFO] [rosbag2_storage]: Opened database 'subset/subset_0.db3' for READ_ONLY.
```

海龟停止运动过后并不会回放过程还在继续，因为还同时记录了/turtle1/pose 话题的消息，rosbag 会在回放完所有的话题消息过后才会退出。

10.3 ROS2 编程【Python】

在此前章节里，我们主要认识了 ROS2 的整体框架和运行环境。本章开始，将以代码工程切实感受 ROS2 最核心的能力--分布式通信机制。无论是机器人上的传感器、执行器，还是后台的算法模块，它们都像一个个“小节点”，通过网络互相交流。听上去抽象，其实就是节点之间在“说话”和“听话”。在这一章，我们会逐步实现这些机制：先学会如何写一个节点，再让节点之间通过话题发布与订阅来传递消息；接着看看如何用服务进行请求与应答；最后还会学习如何通过参数灵活控制节点的行为。

10.3.1 节点与话题

1. 节点 Node

1)ROS2 节点概念

在 ROS2 中单独提出了 graph 概念，在接下来的教程中，将学习构成 ROS2 网络的一系列关于 ROS2 核心概念。ROS 网络一个由 ROS2 中的元素在同一时刻进行数据处理组成的网络。换个可视化的方式来说它包含所有可执行文件以及他们之间的连接情况。

ROS 中的每个节点应该是一个完成特定功能的模块(例如，一个节点用于控制电机，一个节点用于控制激光雷达等等)。每个节点都可以通过话题(topics)、服务(services)、动作库(actions)、参数服务器(parameters)完成数据的发送和接收。一个完整的机器人系统由许多协同工作的节点组成。在 ROS2 中，单个可执行文件(C++程序，Python 程序等)可以包含一个或多个节点。这和 ros1 的一个可执行文件一个节点好像不太一样。

(1)节点的功能和使用—以 TurtleSim 为例

ros2 run

ROS2 中通过 ros2 run 命令从一个包中运行一个可执行文件

```
ros2 run <package_name> <executable_name>
```

例如运行 turtlesim 包中的 turtlesim_node 可执行文件

```
ros2 run turtlesim turtlesim_node
```

ros2 node list

虽然运行了节点，但是并不知道运行节点的名字，通过下面的命令可以查看当前运行的所有节点

```
ros2 node list
```

运行上面命令之后输出如下，表示当前只有一个名叫的/turtlesim 节点

```
/turtlesim
```

再打开一个新的终端运行一个新的节点

```
ros2 run turtlesim turtle_teleop_key
```

我们再次用 ros2 node list 查看节点，输出如下

```
/turtlesim
```

```
/teleop_turtle
```

可以看到现在有两个节点在运行

重映射 Remapping

Remapping 可以对节点、话题、服务等等的名字进行自定义

通过下面命令再打开一个乌龟的界面：

```
ros2 run turtlesim turtlesim_node --ros-args --remap __node:=my_turtle
```

使用 `ros2 node list` 查看，发现多了一个新的节点

```
/my_turtle # 这就是从/turtlesim 重映射出来的新节点
/turtlesim
/teleop_turtle
```

ros2 node info

查看节点信息

```
ros2 node info <node_name>

比如查看

ros2 node info /my_turtle
```

终端返回了许多消息，包含了节点订阅(subscribers)、发布(publishers)、服务(services)、动作(actions)以及 ROS 网络(the ROS graph connections) 中和节点有关的信息

```
/my_turtle

Subscribers:

  /parameter_events: rcl_interfaces/msg/ParameterEvent
  /turtle1/cmd_vel: geometry_msgs/msg/Twist

Publishers:

  /parameter_events: rcl_interfaces/msg/ParameterEvent
  /rosout: rcl_interfaces/msg/Log
  /turtle1/color_sensor: turtlesim/msg/Color
  /turtle1/pose: turtlesim/msg/Pose

Services:

  /clear: std_srvs/srv/Empty
  /kill: turtlesim/srv/Kill
  /reset: std_srvs/srv/Empty
  /spawn: turtlesim/srv/Spawn
  /turtle1/set_pen: turtlesim/srv/SetPen
  /turtle1/teleport_absolute: turtlesim/srv/TeleportAbsolute
  /turtle1/teleport_relative: turtlesim/srv/TeleportRelative
  /my_turtle/describe_parameters: rcl_interfaces/srv/DescribeParameters
  /my_turtle/get_parameter_types: rcl_interfaces/srv/GetParameterTypes
```

```
/my_turtle/get_parameters: rcl_interfaces/srv/GetParameters
/my_turtle/list_parameters: rcl_interfaces/srv/ListParameters
/my_turtle/set_parameters: rcl_interfaces/srv/SetParameters
/my_turtle/set_parameters_atomically: rcl_interfaces/srv/SetParametersAtomically
Action Servers:
  /turtle1/rotate_absolute: turtlesim/action/RotateAbsolute
Action Clients:
```

2)编写自己的一个节点

在 Python 中，节点的实现依赖于 rclpy 这个库，它提供了一个 Node 基类。

当我们自己编写节点时，通常会 **创建一个类并继承 rclpy.node.Node**。

这样做有几个好处：

获得 ROS2 提供的能力：继承以后，我们的类自动具备了 ROS2 节点的“身份”，可以使用话题通信、服务、参数等功能。

构造函数初始化节点：在类的 `__init__` 构造函数中，我们通过 `super().__init__('节点名')` 来完成节点的初始化。括号里的名字，就是这个节点在 ROS2 系统中的标识。

统一管理功能：继承后，我们可以在类中定义方法和属性，把话题发布者、订阅者、服务接口等都组织在一起，保持代码结构清晰。

因此，一个最基本的 ROS2 节点，其核心思想就是：**写一个类，继承 Node，然后在构造函数里定义启动时的行为。**

```
# helloworld_node.py

import rclpy          # ROS2 Python 客户端库，所有 ROS2 程序都要用
from rclpy.node import Node # Node 类是所有节点的基类

# 定义一个自己的节点类，继承自 Node
class HelloWorldNode(Node):
    def __init__(self):
        # 构造函数，初始化节点
        # "hello_world_node" 是节点的名字，启动后在 ROS2 系统中就能看到它
        super().__init__('hello_world_node')
        # 节点启动时打印一句话
        self.get_logger().info("Hello, ROS2 World!")

def main(args=None):
```

```

# 初始化 ROS2 Python 运行环境
rclpy.init(args=args)

# 创建我们自定义的节点对象
node = HelloWorldNode()

# 让节点进入循环，保持运行状态（否则程序会直接退出）
rclpy.spin(node)

# 程序结束时关闭
node.destroy_node()

rclpy.shutdown()

if __name__ == '__main__':
    main()

```

为了快速测试，我们直接只用 python 运行

```
python3 helloworld_node.py
```

终端将会有如下输出

```
[INFO] [168xxx] [hello_world_node]: Hello, ROS2 World!
```

2. 话题

1) 话题介绍

ROS2 将一个复杂的系统分解为由许多模块化的节点组成。话题是 ROS 网络的重要元素，它充当节点交换消息的总线。一个节点可以将数据发布到任意数量的话题，同时也可以订阅任意数量的话题。同 ros1 一样话题是数据在节点之间传递的重要方式之一。

2) 话题基本功能——以 TurtleSim 为例

(1) 启动节点

第一个终端启动 turtlesim_node

```
ros2 run turtlesim turtlesim_node
```

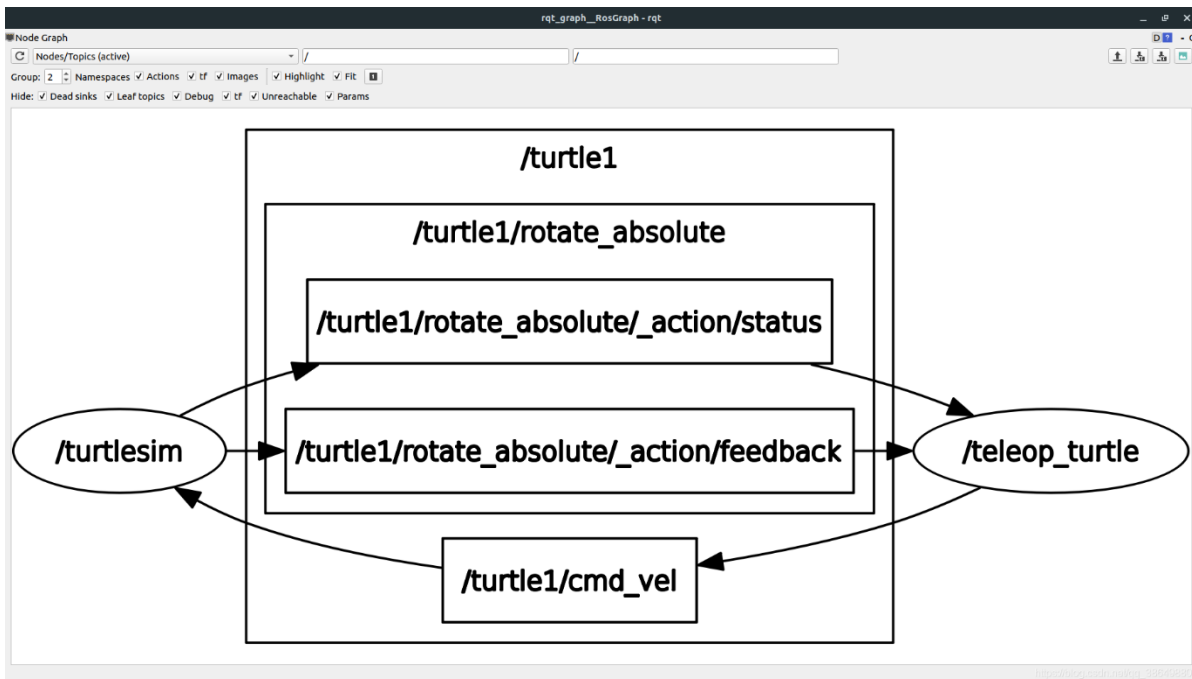
另一个终端启动 turtlesim_teleop_node

```
ros2 run turtlesim turtle_teleop_key
```

(2) rqt_graph

rqt_graph 用于显示话题和节点的连接关系

在终端输入 `rqt_graph` 启动 node graph



我们也可以先运行 `rqt` 然后依次点击 `Plugins > Introspection > Nodes Graph` 也可以打开 `rqt_graph`

(3) `ros2 topic list`

和 `ros2 node list` 类似，`topic list` 会返回当前所有话题列表

```
/parameter_events
/rosout
/turtle1/cmd_vel
/turtle1/color_sensor
/turtle1/pose
```

使用 `ros2 topic list -t` 命令会同时返回节点列表和话题的消息类型

```
/parameter_events [rcl_interfaces/msg/ParameterEvent]
/rosout [rcl_interfaces/msg/Log]
/turtle1/cmd_vel [geometry_msgs/msg/Twist]
/turtle1/color_sensor [turtlesim/msg/Color]
/turtle1/pose [turtlesim/msg/Pose]
```

(4) `ros2 topic echo`

查看 `ros` 话题中当前的发布的消息内容

```
ros2 topic echo <topic_name>
```

比如

```
ros2 topic echo /turtle1/pose
```

(5)ros2 topic info

ros2 中节点可以多对多，也可以多对一，一对多

通过下面的命令我们可以查看当前话题的订阅发布节点的数量

```
ros2 topic info <topic_name>
```

比如

```
ros2 topic info /turtle1/pose
```

终端将会有如下输出

```
Type: turtlesim/msg/Pose # 消息类型是 turtlesim/msg/Pose
```

```
Publisher count: 1 # 目前只有一个节点在发布这个话题
```

```
Subscription count: 0 # 目前没有任何节点订阅这个话题
```

(6)ros2 topic interface show

用于查看消息话题的消息结构

```
ros2 interface show <msg_name>.msg
```

例如

```
ros2 interface show geometry_msgs/msg/Twist.msg
```

终端得到如下结果

```
# This expresses velocity in free space broken into its linear and angular parts.
```

```
Vector3 linear
```

```
Vector3 angular
```

(7)ros2 topic hz

用下面命令查看话题的发布速率

```
ros2 topic hz <topic_name>
```

例如

```
ros2 topic hz /turtle1/pose
```

终端输出发布频率

```
average rate: 63.594
```

```
min: 0.001s max: 0.017s std dev: 0.00193s window: 64
```

```
average rate: 63.050
    min: 0.001s max: 0.017s std dev: 0.00142s window: 127
average rate: 62.858
```

3)消息

(1)消息的介绍

在 ROS2 的通信机制里，节点之间传递的数据被称为**消息（message）**。消息就是一种数据结构，它规定了要传送的内容和格式。比如一个激光雷达节点发布的消息，里面会包含角度、距离等信息；而一个位姿估计节点发布的消息，则可能包含位置和姿态。

ROS2 提供了大量内置的消息类型，这些消息都已经事先定义好，保存在标准接口包里。当我们想知道某个消息的具体结构时，可以使用命令行工具进行查看。例如：

```
ros2 interface show geometry_msgs/msg/PoseStamped
```

(2)官方消息类型

每条消息由若干字段组成，每行用“类型 + 字段名”表示，例如 `float32 x`、`string frame_id`。类型命名遵循固定规则：`int8/16/32/64`、`uint8/16/32/64` 表示不同精度的整数，`float32/float64` 表示浮点数，此外还有 `bool`、`string`、`time`、`duration` 等。消息字段也可以是其他消息类型（如 `geometry_msgs/Pose`），或者数组（如 `float32[] ranges`）。

查看官方文档时，最重要的是“Raw Message Definition”部分，它直接展示 `.msg` 文件原文。以 `PoseStamped` 为例，定义里有两个字段：`Header header` 和 `Pose pose`。其中 `Header` 又包含 `seq`（序号）、`stamp`（时间戳）、`frame_id`（参考坐标系）；而 `Pose` 包含 `Point position`（`x,y,z` 坐标）和 `Quaternion orientation`（`x,y,z,w` 四元数）。这样我们就能理解：`PoseStamped` 其实就是“带时间和坐标系的位姿”。

可以通过查看官方文档确认消息及其字段 [geometry_msgs/PoseStamped Documentation](#)

geometry_msgs/PoseStamped Message

File: `geometry_msgs/PoseStamped.msg`

Raw Message Definition

```
# A Pose with reference coordinate frame and timestamp
Header header
Pose pose
```

Compact Message Definition

```
std_msgs/Header header
geometry_msgs/Pose pose
```

autogenerated on Mon, 28 Apr 2025 02:25:36

std_msgs/Header Message

File: `std_msgs/Header.msg`

Raw Message Definition

```
# Standard metadata for higher-level stamped data types.
# This is generally used to communicate timestamped data
# in a particular coordinate frame.
#
# sequence ID: consecutively increasing ID
uint32 seq
#Two integer timestamps that is expressed as:
# * stamp.sec: seconds (stamp.sec) since epoch (in Python the variable is called 'secs')
# * stamp.nsec: nanoseconds since stamp.sec (in Python the variable is called 'nsecs')
# Time-handling major is provided by the client library
time stamp
#frame this data is associated with
string frame_id
```

Compact Message Definition

```
uint32 seq
time stamp
string frame_id
```

autogenerated on Sun, 13 Apr 2025 02:36:47

geometry_msgs/Pose Message

File: `geometry_msgs/Pose.msg`

Raw Message Definition

```
# A representation of pose in free space, composed of position and orientation.
Point position
Quaternion orientation
```

Compact Message Definition

```
geometry_msgs/Point position
geometry_msgs/Quaternion orientation
```

autogenerated on Mon, 28 Apr 2025 02:25:36

3. 发布者 Publisher

(1)发布者 Publisher 介绍

在 ROS2 中，节点之间的通信主要依赖**发布/订阅（publish/subscribe）机制**。可以把它想象成一个“广播电台”：发布者（Publisher）负责把消息发送出去，就像电台在不断播报新闻；订阅者（Subscriber）则选择性地接收感兴趣的话题，就像听众调到对应频道收听节目。

具体来说，一个节点如果创建了发布者，就会定期或按需向某个****话题（topic）****发送消息；而另一个节点只要订阅了同一个话题，就能自动收到这些消息。两者之间并不需要直接建立连接，ROS2 的通信中间层会负责消息的路由和传输。这种解耦的方式，使得我们可以很方便地增加或移除订阅者，而发布者的逻辑完全不受影响。

因此，**发布者是消息的生产者，订阅者是消息的消费者**。通过话题机制，多个节点就能像拼图一样自由组合，完成传感器数据采集、运动控制、可视化显示等各种功能。

(2)编写发布者代码

我们将编写 **Python 发布者节点**，专门给 `turtlesim` 发布控制指令，让小乌龟先往右走一段，再往上走一段。这里我们使用的话题是 `/turtle1/cmd_vel`，消息类型是 `geometry_msgs/msg/Twist`

```
# turtle_controller.py
```

```

import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
import time

class TurtleController(Node):
    def __init__(self):
        super().__init__('turtle_controller')
        # 创建一个发布者, 话题是 /turtle1/cmd_vel
        self.publisher_ = self.create_publisher(Twist, '/turtle1/cmd_vel', 10)

    def move_turtle(self):
        msg = Twist()
        # 第一步: 向右移动 (x 方向速度为正, 持续 2 秒)
        msg.linear.x = 2.0    # 向前速度
        msg.linear.y = 0.0
        msg.angular.z = 0.0
        self.get_logger().info("Moving right...")
        start_time = time.time()
        while time.time() - start_time < 2.0:
            self.publisher_.publish(msg)
            time.sleep(0.1)
        # 停止一下
        msg.linear.x = 0.0
        self.publisher_.publish(msg)
        time.sleep(1.0)
        # 第二步: 向上移动 (turtlesim 默认坐标系中, 向上=正 y 方向)
        msg.linear.x = 0.0
        msg.linear.y = 2.0    # 在 turtlesim 中, y 方向速度有效
        self.get_logger().info("Moving up...")
        start_time = time.time()
        while time.time() - start_time < 2.0:
            self.publisher_.publish(msg)
            time.sleep(0.1)

```

```

        # 停止
        msg.linear.y = 0.0
        self.publisher_.publish(msg)
        self.get_logger().info("Movement finished.")

def main(args=None):
    rclpy.init(args=args)
    node = TurtleController()
    node.move_turtle()
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

测试:

首先启动 turtlesim

```
ros2 run turtlesim turtlesim_node
```

在另一个终端运行发布者

```
python3 turtle_controller.py
```

发布者语法讲解

在 rclpy 中, 创建发布者使用 `node.create_publisher(...)`。这个方法的核心参数是 消息类型、话题名称和 QoS 配置。调用后会返回一个 `Publisher` 对象, 通常保存为实例属性 (例如 `self.publisher = ...`), 这样就能在类的方法中反复使用它来发送消息。最简示例:

```

self.publisher = self.create_publisher(
    Twist,          # 消息类型
    '/turtle1/cmd_vel', # 话题名
    10              # QoS depth
)

```

发布消息的语法很直接: 先创建消息对象并填充字段, 然后调用 `publisher.publish(msg)`。例如:

```

msg = Twist()
msg.linear.x = 2.0

```

```
msg.angular.z = 0.0
self.publisher.publish(msg)
```

这就把一条速度指令发送到了 `/turtle1/cmd_vel` 话题，所有订阅者都会收到。

和订阅者类似，QoS 参数既可以传一个整数（表示保留的历史消息条数），也可以传完整的 `QoSProfile` 对象，用来细粒度控制消息传输的可靠性、持久性等特性。需要注意的是，只有当发布频率合适时，消息才有实际效果：比如控制小乌龟走直线，通常需要循环定时发布，而不是只发一条就结束。这正是定时器（`create_timer()`）常和发布者搭配使用的原因。

在资源管理上，发布者会跟随节点销毁而释放，不需要手动清理；而如果是长时间运行的控制逻辑，通常会写在回调或定时器里，让消息不断产生。

4. 订阅者 Subscription

(1) 订阅者介绍

在 ROS2 的通信机制里，**订阅者（Subscriber）** 扮演的角色和发布者正好相反。发布者负责把消息“广播”出去，而订阅者则是主动去监听某个话题，只要有新的消息，它就会自动接收并处理。订阅者的核心语法是 `create_subscription()`，其中需要指定话题的 **消息类型、话题名称、回调函数** 以及 QoS 配置。回调函数的作用就是定义“消息到来之后该做什么”，比如把数据打印出来、存入文件，或者作为输入给控制算法。通过发布者和订阅者的协作，ROS2 实现了松耦合的分布式通信，节点之间无需直接依赖，只要共享话题和消息类型，就能完成数据交换。

(2) 编写订阅者订阅 turtlesim 位姿

我们来写一个 **订阅者节点**，去监听 turtlesim 里的 `/turtle1/pose` 话题（消息类型是 `turtlesim/msg/Pose`）。这样节点就能实时打印小乌龟的位姿

```
# turtle_pose_subscriber.py

import rclpy
from rclpy.node import Node
from turtlesim.msg import Pose # 订阅的话题消息类型

class TurtlePoseSubscriber(Node):
    def __init__(self):
        super().__init__('turtle_pose_subscriber')
        # 创建订阅者：指定消息类型、话题名和回调函数
```

```

        self.subscription = self.create_subscription(
            Pose,                    # 消息类型
            '/turtle1/pose',        # 话题名称
            self.listener_callback, # 收到消息时的回调
            10                      # QoS 深度
        )

        self.subscription # 防止未使用的警告

    def listener_callback(self, msg: Pose):
        # 回调函数里处理接收到的 Pose 消息
        self.get_logger().info(
            f"位置: (x={msg.x:.2f}, y={msg.y:.2f}), 朝向: {msg.theta:.2f}, "
            f"线速度: {msg.linear_velocity:.2f}, 角速度: {msg.angular_velocity:.2f}"
        )

def main(args=None):
    rclpy.init(args=args)
    node = TurtlePoseSubscriber()
    rclpy.spin(node) # 保持订阅循环
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

测试:

```

启动 turtlesim:
ros2 run turtlesim turtlesim_node

在另一个终端运行订阅者:
python3 turtle_pose_subscriber.py

再运行 turtle_teleop_key 控制乌龟移动:
ros2 run turtlesim turtle_teleop_key

```

订阅者语法讲解

在 `rclpy` 中创建订阅者主要通过 `node.create_subscription(...)` 完成。常见的调用方式是把消息类型（例如 `Pose`）、话题名（字符串）、回调函数和 QoS（或者一个整数深度）按顺序传入，函数会返回一个 `Subscription` 对象。通常把这个返回值保存为实例属性（例如 `self.subscription = ...`），以防止被垃圾回收导致订阅失效。最简写法示例：

```
self.subscription = self.create_subscription(
    Pose,          # 消息类型
    '/turtle1/pose', # 话题名
    self.listener_callback, # 回调函数（收到消息时调用）
    10             # QoS depth（可用整数快捷写法）
)
```

回调函数的签名通常只接收一个参数 `msg`（即接收到的消息对象），写法例如 `def listener_callback(self, msg):`。在回调里你可以直接访问消息字段（例如 `msg.x, msg.y`），并用 `self.get_logger().info(...)` 输出或把数据传给其他处理模块。需要注意两点：第一，**回调应该尽量非阻塞**——如果需要做耗时计算或 I/O（文件、网络），应把这些工作移到线程池/异步任务或使用多线程执行器，否则会阻塞同一执行器下的其他回调。第二，默认的 `rclpy.spin(node)` 使用单线程执行器，回调按序执行；如果你希望并发处理多个回调，可以使用 `rclpy.executors.MultiThreadedExecutor` 或合理划分 `callback groups`。

关于 QoS 与额外参数：`qos` 参数既可以是一个整数（代表 `history=keep_last, depth=int` 的快捷写法），也可以传入完整的 `QoSProfile` 来精细控制可靠性（`reliable/best_effort`）、持久性（`transient_local`）等。若回调需要额外上下文参数，可以用 `functools.partial` 或 `lambda` 封装，例如 `self.create_subscription(Pose, '/turtle1/pose', lambda m: self.cb(m, other), 10)`。最后，资源清理方面通常由 `node.destroy_node()` 与 `rclpy.shutdown()` 完成，节点销毁时会清理其下的订阅者。

10.3.2 服务

1. 服务的基本概念

(1) 服务的基本概念

在 ROS2 中，服务（Service）是一种重要的通信机制，它实现了节点之间的同步请求-响应模式。想象一下你在餐厅点餐的场景：你（客户端）向服务员（服务端）提出点餐请求，服务员处理你的请求并给你回复。这就是服务通信的基本模式。

服务通信涉及两个角色：

服务端（Server）：提供具体服务功能的节点，等待接收请求，处理后返回响应

客户端（Client）：需要使用服务的节点，发送请求并等待响应

(2)服务与话题的区别

服务和话题的概念易混淆，它们的主要区别是：

话题（Topic）通信特点：

- 异步通信：发布者发布消息后不等待响应，继续执行后续任务
- 一对多通信：一个发布者可以向多个订阅者发送消息
- 持续性通信：适合传感器数据、状态信息等需要持续传输的数据
- 无反馈机制：发布者不知道消息是否被成功接收和处理

服务（Service）通信特点：

- 同步通信：客户端发送请求后会阻塞等待，直到收到响应
- 一对一通信：一个请求只能有一个对应的响应
- 瞬时性通信：适合需要立即反馈的操作，如计算、查询、控制命令
- 有反馈机制：客户端能够确认请求是否被成功处理

(3)服务的适用场景

服务特别适合以下应用场景：

计算服务：需要复杂计算并返回结果，如路径规划、图像处理

查询服务：获取系统状态信息，如传感器标定参数、配置信息

控制服务：执行特定动作并返回执行结果，如机器人移动到指定位置

配置服务：动态修改节点参数或行为模式

文件操作：保存/加载配置文件，数据备份等

2. 服务的工作原理

(1)服务通信流程

服务通信遵循以下步骤：

服务注册：服务端启动时向 ROS2 系统注册自己提供的服务

服务发现：客户端查找所需的服务是否可用

建立连接：客户端与服务端建立通信连接

发送请求：客户端创建请求消息并发送给服务端

处理请求：服务端接收请求，执行相应的处理逻辑

返回响应：服务端将处理结果封装成响应消息返回给客户端

接收响应：客户端接收响应消息，获取处理结果

(2)服务接口定义

每个服务都需要定义其接口，这就像制定 API 规范一样。服务接口通过.srv 文件定义，包含两部分：

```
# 请求部分 (Request)

# 定义客户端向服务端发送的数据格式
请求字段 1 类型
请求字段 2 类型
---

# 响应部分 (Response)

# 定义服务端向客户端返回的数据格式
响应字段 1 类型
响应字段 2 类型
```

示例 - 两数相加服务 AddTwoInts.srv:

```
# 请求：客户端发送两个整数
int64 a
int64 b
---
# 响应：服务端返回计算结果
int64 sum

1
2
3
4
5
6
```

这个定义表示：客户端发送两个 64 位整数 a 和 b，服务端计算它们的和并返回结果 sum。

(3)常用的内置服务类型

ROS2 提供了一些预定义的服务类型，可以直接使用：

```
// 基础服务类型

std_srvs/srv/Empty           // 空服务：无请求参数，无响应数据
std_srvs/srv/SetBool         // 布尔设置：发送布尔值，返回成功状态和消息
std_srvs/srv/Trigger         // 触发服务：无参数触发，返回成功状态和消息

// 示例服务类型
example_interfaces/srv/AddTwoInts    // 两数相加
example_interfaces/srv/SetBool       // 设置布尔参数
```

3. 服务的 Python 实现

1)服务端实现步骤

(1)步骤 1：导入必要的模块

```
#!/usr/bin/env python3

import rclpy                      # ROS2 Python 基础库
from rclpy.node import Node       # 节点基类
from example_interfaces.srv import AddTwoInts # 服务接口
```

(2)步骤 2：创建服务端类

```
class AddTwoIntsServer(Node):

    def __init__(self):
        # 调用父类构造函数，设置节点名称
        super().__init__('add_two_ints_server')

        # 创建服务
        # 参数说明：
        # - AddTwoInts：服务类型
```

```

# - 'add_two_ints': 服务名称

# - self.handle_service: 处理请求的回调函数
self.srv = self.create_service(
    AddTwoInts,
    'add_two_ints',
    self.handle_service
)

self.get_logger().info('加法服务端已启动，正在等待客户端请求...')

def handle_service(self, request, response):
    """
    处理服务请求的回调函数

    参数:
        request: 客户端发送的请求数据
        response: 需要填充并返回给客户端的响应数据

    返回:
        response: 填充完毕的响应数据
    """
    # 执行加法计算
    response.sum = request.a + request.b

    # 记录日志，便于调试和监控
    self.get_logger().info(
        f'收到请求: {request.a} + {request.b}, 计算结果: {response.sum}'
    )

    # 必须返回 response 对象
    return response

```

(3)步骤 3: 主函数

```

def main(args=None):
    # 初始化 ROS2 Python 客户端库
    rclpy.init(args=args)

    # 创建服务端节点
    server = AddTwoIntsServer()

    try:
        # 进入事件循环，持续监听和处理请求
        rclpy.spin(server)
    except KeyboardInterrupt:
        # 用户按 Ctrl+C 时优雅退出
        print('\n 程序被用户中断')
    finally:
        # 清理资源
        server.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

(4)客户端实现步骤

步骤 1: 创建客户端类

```

class AddTwoIntsClient(Node):
    def __init__(self):
        super().__init__('add_two_ints_client')

        # 创建服务客户端
        self.client = self.create_client(AddTwoInts, 'add_two_ints')

        # 等待服务变为可用
        # 这是一个重要步骤！确保服务端已经启动
        timeout_counter = 0

```

```

while not self.client.wait_for_service(timeout_sec=1.0):
    timeout_counter += 1
    self.get_logger().info(f'等待服务可用... ({timeout_counter}秒)')

    # 避免无限等待
    if timeout_counter > 10:
        self.get_logger().error('服务长时间不可用，请检查服务端是否启动')
        return

self.get_logger().info('服务已连接，可以发送请求')

def send_request(self, a, b):
    """
    发送服务请求

    参数:
        a, b: 要相加的两个数字

    返回:
        Future 对象，用于异步获取响应
    """
    # 创建请求对象
    request = AddTwoInts.Request()
    request.a = a
    request.b = b

    self.get_logger().info(f'发送请求: {a} + {b}')

    # 异步发送请求
    future = self.client.call_async(request)
    return future

```

步骤 2: 主函数实现

```
import sys # 用于处理命令行参数

def main(args=None):
    rclpy.init(args=args)

    # 检查命令行参数
    if len(sys.argv) != 3:
        print('错误: 参数数量不正确')
        print('用法: ros2 run package_name client X Y')
        print('例如: ros2 run package_name client 10 20')
        return

    try:
        # 将命令行参数转换为整数
        num1 = int(sys.argv[1])
        num2 = int(sys.argv[2])
    except ValueError:
        print('错误: 参数必须是整数')
        return

    # 创建客户端
    client = AddTwoIntsClient()

    try:
        # 发送请求
        future = client.send_request(num1, num2)

        # 等待响应
        rclpy.spin_until_future_complete(client, future)

        # 检查响应结果
        if future.result() is not None:
            response = future.result()
            client.get_logger().info(
                f'收到响应: {num1} + {num2} = {response.sum}'
            )
```



```

        )
    else:
        client.get_logger().error('服务调用失败, 未收到响应')

except KeyboardInterrupt:
    print('\n 程序被用户中断')
finally:
    client.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

(5)Python 中的同步调用

Python 也支持同步调用，但需要小心使用：

```

def sync_call_service(self, a, b):
    """
    同步服务调用示例
    注意：这会阻塞程序直到收到响应或超时
    """
    request = AddTwoInts.Request()
    request.a = a
    request.b = b

    try:
        # 同步调用，最多等待 5 秒
        response = self.client.call(request, timeout_sec=5.0)
        self.get_logger().info(f'同步调用结果: {response.sum}')
        return response
    except Exception as e:
        self.get_logger().error(f'服务调用失败: {e}')
        return None

```

4. 项目配置详解

Python 项目配置要点

(1)setup.py 核心修改:

```
entry_points={
    'console_scripts': [
        # 这里定义了可以通过 ros2 run 运行的命令
        'server = service_example_py.server:main',
        'client = service_example_py.client:main',
    ],
},
```

(2)package.xml 核心修改:

```
<!-- Python 项目的依赖 -->
<exec_depend>rclpy</exec_depend>
<exec_depend>example_interfaces</exec_depend>
```

(3)Python 项目结构:

```
service_example_py/
├── package.xml
├── setup.py
├── setup.cfg
├── resource/
│   └── service_example_py
└── service_example_py/
    ├── __init__.py
    ├── server.py
    └── client.py
```

5. 编译运行实战

1)完整工作流程

(1)第一步: 创建工作空间

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws/src
```

(2)第二步：创建包

```
# Python 包
ros2 pkg create --build-type ament_python service_example_py --dependencies rclpy
example_interfaces
```

(3)第三步：编写代码

将前面的服务端和客户端代码分别放入对应的文件中。

(4)第四步：编译

```
cd ~/ros2_ws
colcon build --packages-select service_example_cpp service_example_py
```

(5)第五步：测试运行

```
# 激活环境
source install/setup.bash

# 第一个终端运行服务端
ros2 run service_example_cpp server

# 第二个终端运行客户端
ros2 run service_example_cpp client 10 20
```

(6)常用调试命令

(7)查看服务状态：

```
# 列出所有可用服务
ros2 service list

# 查看服务类型
ros2 service type /add_two_ints

# 查看服务接口定义
```

```
ros2 interface show example_interfaces/srv/AddTwoInts

# 直接调用服务进行测试
ros2 service call /add_two_ints example_interfaces/srv/AddTwoInts "{a: 15, b: 25}"
```

(8)查看节点信息:

```
# 列出运行中的节点
ros2 node list

# 查看特定节点信息
ros2 node info /add_two_ints_server

# 查看计算图
rqt_graph
```

10.3.3 参数

在 ROS 2 中，参数（Parameters）是节点配置的重要方式。它们允许用户在运行时调整节点的行为，而无需重新编译代码或重启节点。参数可以是各种数据类型，如整数、浮点数、字符串等，并且支持动态设置和获取。

1. 参数的基本概念

参数服务器：在 ROS 2 中，每个节点都有自己的参数服务器，这意味着参数是与特定节点关联的。

参数类型：支持多种数据类型，包括整型(int)、浮点型(float/double)、布尔型(bool)、字符串(string)以及数组(array)等。

参数作用域：参数分为全局参数和局部参数（私有参数）。私有参数以节点名称作为前缀，用于防止命名冲突并限定其使用范围。

2. 如何在节点中使用参数

以下是如何在 ROS 2 Python 节点中声明、获取和设置参数的基本步骤：

声明参数

```
import rclpy
```

```

from rclpy.node import Node

from rcl_interfaces.msg import SetParametersResult

class MinimalParamNode(Node):

    def __init__(self):
        super().__init__('minimal_param_node')

        # 1. 声明参数

        self.declare_parameter('counter_start', 0)

```

获取参数值

```

example_param_value =
self.get_parameter('counter_start').get_parameter_value().integer_value

```

动态设置参数

使用 ROS 2 命令行工具 `ros2 param set` 可以动态地修改正在运行中的节点参数值。

例如，在终端中运行如下命令来更改 `example_param` 的值：

```
ros2 param set /minimal_param_node counter_start 100
```

监听参数变化

监听某个参数的变化并在变化时执行特定操作，注册回调函数：

```

def param_change_callback(self, params):
    result = SetParametersResult()
    result.successful = True

    for param in params:
        if param.name == 'counter_start':
            self.get_logger().info(f"参数变更: {param.name} = {param.value}")
    return result

# 3. 添加回调

self.callback_handle = self.add_on_set_parameters_callback(self.param_change_callback)

```

3. 使用示例

下面是一个完整的例子，展示了如何在一个简单的 ROS 2 节点中使用参数：

```

import rclpy
from rclpy.node import Node
from rcl_interfaces.msg import SetParametersResult

class MinimalParamNode(Node):
    def __init__(self):
        super().__init__('minimal_param_node')

        # 1. 声明参数
        self.declare_parameter('counter_start', 0)

        # 2. 注册回调
        def callback(params):
            result = SetParametersResult()
            result.successful = True
            for param in params:
                if param.name == 'counter_start':
                    self.get_logger().info(f"参数变更: {param.name} = {param.value}")
            return result

        # 3. 添加回调
        self.callback_handle = self.add_on_set_parameters_callback(callback)

def main(args=None):
    rclpy.init(args=args)

    # 模拟 C++ 中的日志初始化部分
    print("=====main=====")
    node = MinimalParamNode()
    rclpy.spin(node)
    print("----end----")
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':

```

10.4 ROS2 仿真

10.4.1 机器人建模与仿真概述

在机器人系统开发过程中，仿真环境是连接算法设计与真实硬件之间的重要桥梁。通过仿真平台，开发者可以在不依赖实体机器人的情况下验证运动控制、传感器处理与导航算法，从而降低实验成本并提高调试效率。ROS2 生态中提供了较为完善的建模与仿真工具链，使机器人可以先在虚拟环境中完成结构建模、传感器配置与行为测试，再逐步迁移到真实系统。

在仿真流程中，通常首先需要建立机器人的结构模型，用于描述连杆关系、关节类型、尺寸参数与惯性属性等信息。在此基础上，仿真引擎根据物理参数计算运动行为，并结合虚拟传感器插件生成激光雷达、相机或里程计等数据话题，供 ROS2 节点订阅和处理。因此，从 ROS2 角度看，仿真系统与真实机器人在接口形式上尽量保持一致，这也是算法可以在仿真与实机之间迁移的重要前提。

常见流程是先完成机器人模型描述，再加载到仿真引擎中运行，通过 `rviz` 与仿真窗口联合观察机器人状态与传感器数据。图 10-x 建议展示“仿真环境 + `rviz` 可视化同步显示”的整体效果。

10.4.2 使用 URDF 创建机器人

在 ROS2 中，机器人结构通常使用 URDF（Unified Robot Description Format）进行描述。URDF 是一种基于 XML 的机器人模型描述格式，用于定义机器人各个连杆与关节之间的拓扑关系，以及基本的几何、惯性和碰撞属性。通过 URDF 文件，可以将机器人从机械结构层面抽象为软件模型，使系统能够自动构建坐标变换关系与运动学结构。

一个基本的 URDF 模型通常包含连杆与关节两类核心元素。连杆用于描述刚体部分，可以指定形状尺寸、可视化模型与碰撞模型；关节用于描述连杆之间的连接关系和运动类型，例如固定关节、旋转关节或滑动关节。通过逐级连接，可以构建出完整的机器人结构树。模型建立完成后，可以通过 `robot_state_publisher` 等节点发布关节状态与坐标变换关系，并在 `rviz` 中直接可视化机器人模型姿态。

在实际开发中，URDF 往往配合 `xacro` 宏文件使用，以减少重复结构描述并支持参数化生成模型。教学实践中可以先从简单两连杆模型入手，逐步扩展到带底盘与传感器的完整移动机器人结构。图 10-x 建议配“URDF 结构示意图 + `rviz` 中显示机器人模型”的对照图。

1. URDF 基本结构说明

URDF（Unified Robot Description Format，统一机器人描述格式）是 ROS 与 ROS2 中用于描述机器人结构的标准文件格式，本质上是一种基于 XML 的描述语言。URDF 文件的作用是从结构层面定义机器人由哪些刚体部件构成，这些部件之间通过什么方式连接，以及各部分的几何外形、碰撞模型和物理参数。在可视化、仿真、导航与建图等系统中，URDF 都作为机器人模型的基础输入被加载和解析。URDF 本身并不包含控制算法或行为逻辑，它只负责描述机器人“长什么样”和“各部件之间如何连接”。

一个 URDF 文件最外层必须使用 `robot` 标签作为整体容器，在该标签内部依次定义多个 `link` 和 `joint` 元素。可以把 `link` 理解为刚体零件，把 `joint` 理解为零件之间的连接约束关系。ROS2 在读取 URDF 后，会根据这些定义自动构建一棵从根节点开始的坐标变换树，并在 TF 系统中发布各坐标系之间的关系。因此，从工程角度看，URDF 同时也是机器人坐标体系结构的来源。

下面给出一个最小但语法完整的 URDF 示例模型。该模型包含一个底座和一个上方的立柱，通过一个固定关节连接，结构简单但包含完整的 `visual`、`collision` 和 `inertial` 信息，能够直接用于 RViz 显示和 Gazebo 仿真。

```
<?xml version="1.0"?>
<robot name="simple_bot">

  <link name="base_link">
    <visual>
      <geometry>
        <box size="0.4 0.3 0.1"/>
      </geometry>
      <origin xyz="0 0 0.05" rpy="0 0 0"/>
      <material name="blue">
        <color rgba="0 0 0.8 1"/>
      </material>
    </visual>

    <collision>
      <geometry>
        <box size="0.4 0.3 0.1"/>
      </geometry>
      <origin xyz="0 0 0.05" rpy="0 0 0"/>
```



```
</collision>

<inertial>
  <mass value="5.0"/>
  <origin xyz="0 0 0.05"/>
  <inertia ixx="0.02" ixy="0" ixz="0"
    iyy="0.02" iyz="0"
    izz="0.02"/>
</inertial>
</link>

<link name="sensor_link">
  <visual>
    <geometry>
      <cylinder radius="0.03" length="0.3"/>
    </geometry>
    <origin xyz="0 0 0.15" rpy="0 0 0"/>
    <material name="gray">
      <color rgba="0.6 0.6 0.6 1"/>
    </material>
  </visual>

  <collision>
    <geometry>
      <cylinder radius="0.03" length="0.3"/>
    </geometry>
    <origin xyz="0 0 0.15" rpy="0 0 0"/>
  </collision>

  <inertial>
    <mass value="0.5"/>
    <origin xyz="0 0 0.15"/>
    <inertia ixx="0.001" ixy="0" ixz="0"
      iyy="0.001" iyz="0"
      izz="0.001"/>
  </inertial>
</link>
```

```

    </inertial>
  </link>

  <joint name="sensor_joint" type="fixed">
    <parent link="base_link"/>
    <child link="sensor_link"/>
    <origin xyz="0 0 0.1" rpy="0 0 0"/>
  </joint>

</robot>

```

在 URDF 中，link 表示机器人的一个刚性部件，每个 link 都拥有自己的局部坐标系。link 通常包含三类信息：visual、collision 和 inertial。visual 用于可视化显示，决定在 RViz 或 Gazebo 中看到的外形；collision 用于碰撞检测，通常可以使用更简单的几何体以提高计算效率；inertial 用于动力学仿真，描述该刚体的质量和转动惯量。在只做模型显示时，惯性参数可以使用近似值，但在物理仿真环境中应尽量保证合理，否则会导致运动行为异常。

在实际机器人系统中，通常会约定一个名为 base_link 的主体坐标系，作为整机的参考坐标原点。导航、定位与运动控制模块往往默认以 base_link 作为机器人本体姿态参考。从语法角度看，base_link 只是一个普通的 link 名称，并不是保留关键字，但在工程实践中已经形成通用约定。一般会把底盘几何中心或质心附近设为 base_link 原点，这样可以减少后续坐标变换和外参标定的复杂度。

joint 用于描述两个 link 之间的连接方式，同时定义它们之间的相对位姿关系。每个 joint 必须明确指定 parent 和 child，从而把多个 link 连接成一棵无环的树状结构。URDF 不允许出现闭环拓扑，因此所有结构必须能够从根 link 出发逐级展开。joint 中的 origin 参数表示子坐标系相对于父坐标系的位姿偏移，xyz 表示平移，rpy 表示欧拉角旋转，这个定义直接决定了 TF 坐标树中的变换关系。

joint 还需要指定类型，例如 fixed 表示刚性固定连接，revolute 表示绕某一轴旋转的关节，prismatic 表示沿某一轴滑动的关节，continuous 表示无角度限制的连续旋转关节。在可动关节中，还可以进一步定义运动轴方向和角度或位移范围，从而为运动学与动力学计算提供约束条件。通过 link 与 joint 的逐级连接，就可以完整描述机械臂、多连杆机构或移动机器人本体结构，并被 ROS2 的可视化、仿真和导航系统直接使用。

2. 加载 URDF 并在 Rviz 中可视化

URDF 文件本身是一个描述文件，并不需要像源代码那样“编译”，但在 ROS2 工作流中，它通常会被放入功能包中，通过 launch 文件加载，并交给 robot_state_publisher 与 RViz 进行解析和可视化。从教

材写作角度，可以把“编译并使用可视化”表述为：将 URDF 集成到 ROS2 功能包 → 构建工作空间 → 通过节点发布模型 → 在 RViz 中显示模型。下面用一套完整流程把这件事按工程实践方式讲清楚，正文可直接用，命令块可保留。

在 ROS2 中使用 URDF 的第一步通常是建立一个专门存放模型的功能包。推荐使用 `ament_python` 或 `ament_cmake` 都可以，如果只是放模型文件，用 `ament_python` 更轻量。创建功能包后，把 `urdf` 文件放入独立目录，形成清晰结构，便于后续被 `launch` 文件和仿真系统加载。

首先在工作空间 `src` 目录下创建功能包。

```
cd ~/ros2_ws/src
ros2 pkg create my_robot_description --build-type ament_python
```

在功能包中创建 `urdf` 目录，并把模型文件放进去。

```
cd my_robot_description
mkdir urdf
vim urdf/simple_bot.urdf
```

把之前写好的 URDF 内容粘贴进去保存即可。为了让 ROS2 在运行时能找到这些资源文件，需要在 `setup.py` 中把 `urdf` 目录加入安装项，否则构建后不会被安装到 `share` 路径下。可以在 `setup.py` 的 `data_files` 中加入一行，把 `urdf` 目录整体安装。

```
data_files=[
    ('share/ament_index/resource_index/packages',
     ['resource/my_robot_description']),
    ('share/my_robot_description', ['package.xml']),
    ('share/my_robot_description/urdf', ['urdf/simple_bot.urdf']),
],
```

随后返回工作空间根目录进行构建，并刷新环境。

```
cd ~/ros2_ws
colcon build
source install/setup.bash
```

URDF 的可视化依赖一个关键节点 `robot_state_publisher`。这个节点会读取 URDF，解析 `link` 与 `joint` 结构，并向 TF 系统持续发布坐标变换关系。RViz 正是通过 TF 与 `robot_description` 参数来显示机器人模型。因此只要正确启动 `robot_state_publisher`，就已经完成了“模型发布”。

可以直接用命令行方式加载 URDF 并启动该节点。

```
ros2 run robot_state_publisher robot_state_publisher urdf/simple_bot.urdf
```

但在实际工程中更标准的做法是写一个 `launch` 文件，把 URDF 作为参数传入。可以在功能包中创建 `launch` 目录并新建 `launch` 文件。

```
mkdir launch
vim launch/display.launch.py
```

写入如下内容：

```
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.substitutions import Command
from launch_ros.parameter_descriptions import ParameterValue
from ament_index_python.packages import get_package_share_directory
import os

def generate_launch_description():
    pkg_path = get_package_share_directory('my_robot_description')
    urdf_path = os.path.join(pkg_path, 'urdf', 'simple_bot.urdf')

    robot_desc = ParameterValue(
        Command(['cat ', urdf_path]),
        value_type=str
    )

    return LaunchDescription([
        Node(
            package='robot_state_publisher',
            executable='robot_state_publisher',
            parameters=[{'robot_description': robot_desc}]
        )
    ])

```

重新构建一次工作空间后，就可以通过 `launch` 文件启动模型发布节点。

```
colcon build
source install/setup.bash
ros2 launch my_robot_description display.launch.py
```

接下来启动 RViz 进行可视化。RViz 是 ROS2 中最常用的三维可视化工具，可以显示机器人模型、TF 坐标系、传感器数据与地图等信息。

rviz2

RViz 打开后，需要手动添加机器人模型显示组件。在左侧 Displays 面板点击 Add，选择 RobotModel。添加后，把 Description Source 保持为参数方式（默认读取 robot_description），此时如果 robot_state_publisher 正在运行，就会看到机器人模型已经被加载显示。如果模型位置异常，可以把 Fixed Frame 设置为 base_link 或模型的根坐标系名称。

如果需要观察坐标系关系，可以再添加一个 TF 显示组件，用来显示各 link 坐标轴，这对于检查 joint 原点位置和方向是否正确非常有帮助。

可以把这一整套流程理解为：URDF 负责定义结构，robot_state_publisher 负责把结构变成坐标变换发布到系统中，RViz 负责把这些结构与坐标关系以三维形式显示出来。只要这三个环节打通，就完成了 URDF 模型从文件到可视化的完整使用链路。

10.4.3 Gazebo 仿真

Gazebo 是 ROS2 体系中常用的三维物理仿真平台，用于在虚拟环境中对机器人模型进行运动与传感器仿真。在前一小节中，URDF 模型已经可以通过 robot_state_publisher 在 RViz 中进行结构可视化，但 RViz 只负责显示模型与坐标变换，并不提供真实物理引擎。而 Gazebo 提供了重力、碰撞、摩擦、动力学与传感器模拟能力，可以在不使用真实硬件的情况下对机器人系统进行功能验证与算法调试。

在 ROS2 中使用 Gazebo 仿真时，通常需要完成三个步骤：为 URDF 增加仿真相关属性，准备 Gazebo 启动环境，通过 launch 文件把机器人模型加载到仿真世界中运行。整体流程与前面 URDF 可视化类似，但多了一层物理与世界环境的加载过程。

URDF 模型如果要在 Gazebo 中参与物理仿真，需要具备基本的惯性参数与碰撞模型定义，也就是每个主要 link 至少应包含 inertial 与 collision 标签，否则模型可能可以显示，但无法参与物理计算或会出现异常行为。在此基础上，还可以通过 gazebo 扩展标签为 link 指定摩擦系数、材质与传感器插件等仿真属性。例如可以为底盘 link 增加 Gazebo 表面参数，使其与地面接触时具有合理摩擦效果。

```
<gazebo reference="base_link">
  <mu1>0.8</mu1>
  <mu2>0.8</mu2>
</gazebo>
```

完成模型准备后，需要确保系统已安装 ROS2 对应版本的 Gazebo 集成包。不同 ROS2 发行版通常配套 gazebo_ros 或 ros_gz 系列桥接包，用于把 Gazebo 与 ROS2 通信系统连接起来，使仿真中的机器人能够发布话题与 TF。

Gazebo 一般通过 ROS2 launch 文件启动，这样可以同时启动仿真世界与机器人模型。典型做法是先启动 Gazebo 空世界，再通过 spawn_entity 工具把 URDF 模型插入到仿真环境中。可以在功能包中新建一个仿真 launch 文件，将模型路径与生成节点写入其中。

```
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.actions import ExecuteProcess
from ament_index_python.packages import get_package_share_directory
import os

def generate_launch_description():
    pkg_path = get_package_share_directory('my_robot_description')
    urdf_path = os.path.join(pkg_path, 'urdf', 'simple_bot.urdf')

    gazebo = ExecuteProcess(
        cmd=['gazebo', '--verbose'],
        output='screen'
    )

    spawn = Node(
        package='gazebo_ros',
        executable='spawn_entity.py',
        arguments=[
            '-entity', 'simple_bot',
            '-file', urdf_path
        ],
        output='screen'
    )

    return LaunchDescription([
        gazebo,
```

```
spawn
```

```
}}
```

构建工作空间并加载环境后，就可以通过 `launch` 文件启动仿真。

```
colcon build
```

```
source install/setup.bash
```

```
ros2 launch my_robot_description gazebo.launch.py
```

Gazebo 启动后会打开三维仿真窗口，默认包含地面与光源环境，机器人模型会被自动加载到世界坐标原点位置。此时可以通过鼠标在仿真界面中移动视角，观察机器人结构。如果模型具有可动关节，并且加载了控制插件，还可以进一步通过 ROS2 话题或控制器驱动机器人运动。

在工程实践中，Gazebo 仿真通常与 RViz 配合使用：Gazebo 负责物理世界与传感器仿真，RViz 负责算法结果与感知数据可视化。例如可以在 Gazebo 中运行激光雷达插件，同时在 RViz 中显示激光点云与地图结果。这种“仿真 + 可视化”双窗口方式是 ROS2 开发中最常见的调试模式之一。

10.5 ROS2 导航和建图

10.5.1 机器人导航介绍

机器人导航系统的目标是在已知或未知环境中实现自主移动，使机器人能够从当前位置安全到达目标位置。一个完整的导航系统通常包括定位、建图、路径规划与运动控制等多个模块，这些模块通过统一的坐标系与数据接口协同工作。在 ROS2 体系中，导航相关功能由多个标准化功能包共同构成，开发者可以按需组合使用。

从功能流程角度看，导航系统通常先依赖传感器数据进行环境感知与地图构建，然后通过定位算法估计机器人在地图中的位姿，再由路径规划模块计算可行路径，最后由控制器将路径转换为底盘速度指令。为了便于调试，这些中间结果通常都可以在 rviz 中进行可视化展示。图 10-x 建议给出导航系统模块关系示意图。

10.5.2 使用 slamtoolbox 建图

在未知环境中运行机器人时，通常需要通过 SLAM（即时定位与建图）方法一边移动一边构建地图。slam_toolbox 是 ROS2 中常用的二维激光 SLAM 工具包，适合教学与实验环境使用。它能够订阅激光雷达与里程计数据，实时生成栅格地图，并持续优化位姿估计结果。

`slam_toolbox` 软件包已经过基准测试，可在 5 倍以上实时速度构建约 30,000 平方英尺的地图，在 3 倍实时速度构建约 60,000 平方英尺的地图。使用的最大面积约 200,000 平方英尺。以同步模式建图（即处理所有激光点云，无论延迟如何），并以异步模式构建更大的空间。

建图过程概述如下：

ROS 节点：SLAM toolbox 以同步模式运行，生成 ROS 节点。该节点订阅激光扫描点云和里程计主题，并发布 TF(地图到里程计的变换)和地图。

获取里程计和激光雷达数据：激光主题订阅的回调函数中，将生成一个位姿（使用里程计）和绑定在该节点的激光扫描。这些 `PosedScan` 对象形成一个队列。

数据处理：`PosedScan` 对象队列用于构建位姿图；使用激光扫描匹配来完善里程计。该位姿图用于计算机器人位姿，并找到闭环。如果发现闭环，则优化位姿图，并更新姿态估计。姿态估计用于计算和发布机器人的里程计到地图坐标系的变换。

建图：与位姿图中每个位姿相关的激光扫描用于构建和发布地图。

1. Slamtoolbox 启动建图

`slam_toolbox` 提供了现成的启动文件，可以直接以在线建图模式运行。在 ROS2 环境加载完成后，可以使用如下命令启动在线异步建图节点。

```
ros2 launch slam_toolbox online_async_launch.py
```

节点启动后会订阅激光数据与 TF 信息，并在内部持续进行扫描匹配与位姿优化计算。与此同时，可以启动 RViz 进行可视化观察。RViz 中需要添加 Map、LaserScan 和 TF 显示组件，其中 Map 组件订阅 `/map` 话题，LaserScan 组件订阅 `/scan` 话题，即可看到地图随机器人运动逐步扩展的过程。教材中建议在此处配一张“机器人移动—地图逐步生成”的连续截图序列。

在实际建图过程中，机器人需要以较慢速度平稳移动，尽量减少急转弯和剧烈抖动，这有助于提高匹配稳定性与地图质量。对于室内环境，通常采用“沿墙走一圈，再穿插走廊”的覆盖式路径策略，可以减少回环误差。`slam_toolbox` 内部带有回环检测与图优化机制，当机器人回到曾经到达过的区域时，会自动进行全局一致性修正。

当地图构建完成后，可以使用地图保存工具将结果导出为地图文件，供后续导航系统使用。ROS2 中通常通过 `nav2_map_server` 提供的保存工具完成该操作。

```
ros2 run nav2_map_server map_saver_cli -f my_map
```


执行后会在当前目录生成地图图像文件与对应的 YAML 配置文件，这两部分共同构成导航系统使用的静态地图数据。图像文件保存栅格占据情况，YAML 文件记录分辨率与坐标原点等元信息。在后续 Navigation2 导航系统中，只需加载该 YAML 文件即可完成地图输入。

slam_toolbox 还支持定位模式运行，即在已有地图基础上只进行位姿估计而不再更新地图，这种模式适合“先建图、再导航”的两阶段流程。在教学实验中，通常先进行一次完整建图并保存地图，再在导航实验中切换为定位模式运行，从而保持地图稳定不被重复修改。

2. Slamtoolbox 主要 API

订阅的主题

Topic	Type	描述
/scan	sensor_msgs/LaserScan	来自激光雷达的输入扫描数据
tf	N/A	配置的 odom_frame 到 base_frame 的转换

发布的主题

Topic	Type	描述
map	nav_msgs/OccupancyGrid	以 map_update_interval 频率表示位姿图的占用网格
pose	geometry_msgs/PoseWithCovarianceStamped	配置的 map_frame 中 base_frame 的位姿以及根据扫描匹配计算的协方差

发布的服务

Topic	Type	描述
/slam_toolbox/clear_changes	slam_toolbox/Clear	清除所有待处理的手动位姿图操作的更改
/slam_toolbox/deserialize_map	slam_toolbox/DeserializePoseGraph	从磁盘加载保存的序列化位姿图文件
/slam_toolbox/dynamic_map	nav_msgs/OccupancyGrid	请求位姿图的当前状态作为占用网格
/slam_toolbox/manual_loop_closure	slam_toolbox/LoopClosure	请求对位姿图进行手动更改
/slam_toolbox/pause_new_measurements	slam_toolbox/Pause	暂停处理新传入的激光扫描
/slam_toolbox/save_map	slam_toolbox/SaveMap	保存可用于显示或 AMCL 定位的地图图像文件。它是 map_server/map_saver 上的一个简单包装器，但很有用。
/slam_toolbox/serialize_map	slam_toolbox/SerializePoseGraph	保存地图位姿图和数据，可用于继续建图、slam_toolbox 定位、离线操作等
/slam_toolbox/toggle_interactive_mode	slam_toolbox/ToggleInteractive	切换进入和退出交互模式

10.5.3 导航框架 Navigation2

Navigation2（通常简称 Nav2）是 ROS2 中标准的移动机器人导航框架，用于实现机器人在已知地图中的自主路径规划与避障移动。该框架是在 ROS1 导航栈基础上的重构版本，针对 ROS2 的通信机制、生命周期节点与行为树调度模型进行了重新设计，具有更好的模块化程度与可扩展性。在实际系统中，

Navigation2 通常与前一阶段通过 `slam_toolbox` 构建的地图文件配合使用，从而形成“建图—定位—导航”的完整自主移动能力链条。

从功能结构上看，Navigation2 并不是单一节点，而是一组协同工作的功能模块集合，其中包括全局路径规划器、局部控制器、代价地图模块、行为树调度器以及恢复行为模块等。系统接收目标点位姿作为输入，根据当前地图与机器人位姿计算全局路径，再由局部控制器结合实时传感器数据进行动态避障与速度控制输出。整个导航过程由行为树统一调度，因此导航行为具有清晰的状态结构与可恢复机制。

在运行 Navigation2 之前，需要满足几个基本条件：系统已经具备机器人 URDF 模型与 TF 变换关系；底盘里程计与激光雷达数据能够正常发布；已经通过 SLAM 建图得到地图 YAML 文件；机器人定位模块可以提供 map 到 base_link 的位姿关系。在教学实践中，通常通过仿真机器人平台先完成这套条件准备，再进行导航实验。

Navigation2 提供了标准启动方式，可以直接加载地图文件与参数配置启动完整导航系统。典型启动命令如下所示，其中 map 参数指向前一节保存的地图 YAML 文件。

10.6

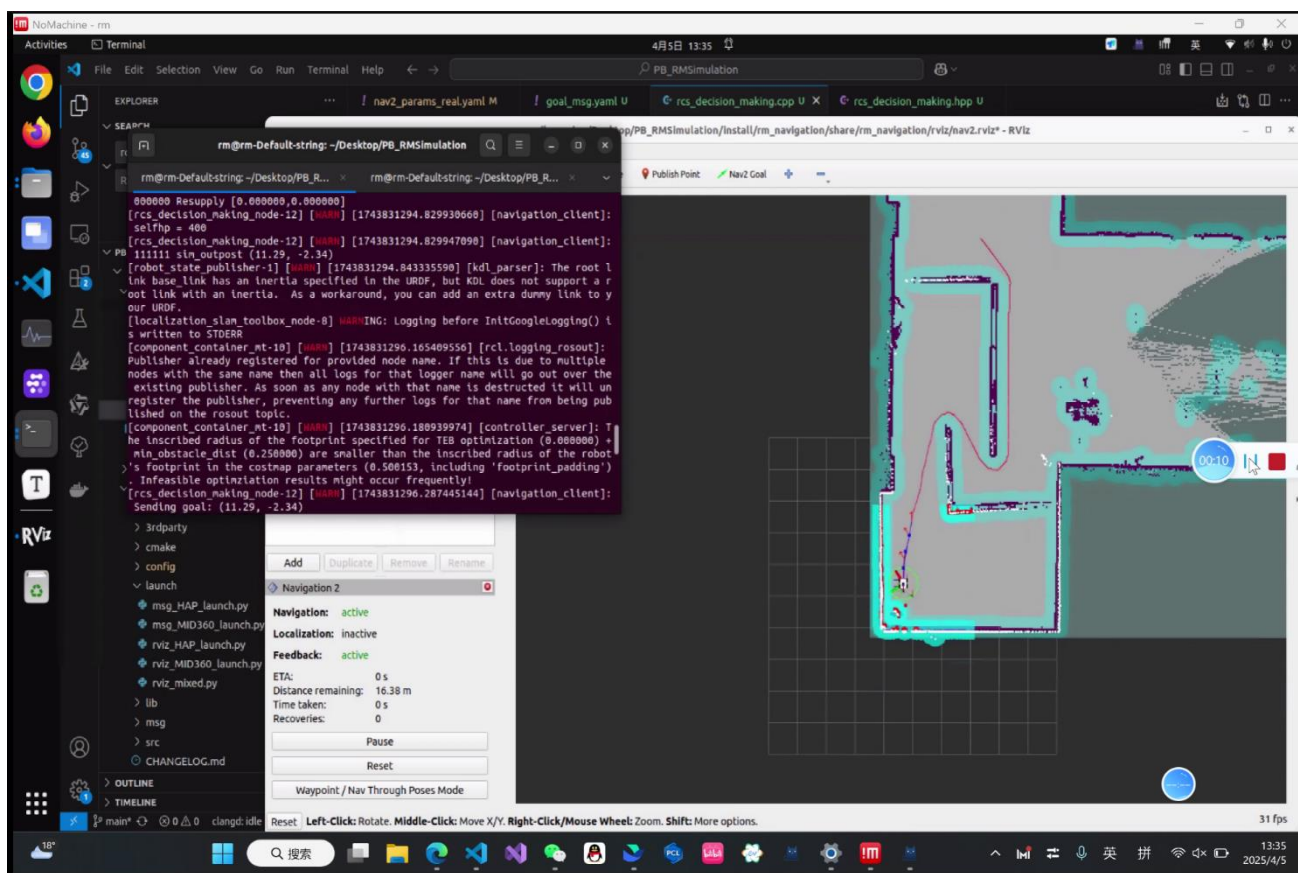
```
ros2 launch nav2_bringup navigation_launch.py map:=/home/user/my_map.yaml
```

系统启动后会同时加载地图服务器、定位模块、规划器、控制器与行为树导航节点。此时可以打开 RViz 进行交互式导航测试。在 RViz 中加载 Nav2 配套配置后，可以看到地图、机器人模型、代价地图与规划路径等显示层。通过工具栏中的“2D Goal Pose”工具在地图上点击目标位置并设定朝向，系统就会自动计算路径并驱动机器人向目标点移动。

在导航运行过程中，可以在 RViz 中观察到多种关键可视化信息，包括全局路径曲线、局部避障轨迹、局部代价地图膨胀区域等。这些信息对于理解导航系统内部工作机制非常重要。教材中建议配图展示“目标点设定界面”“规划路径显示效果”“机器人绕障碍物行走过程”三类典型画面，有助于读者建立直观认知。

Navigation2 的参数系统高度可配置，规划算法、代价地图分辨率、障碍物膨胀半径、控制器类型等都可以通过 YAML 参数文件进行调整。在教学入门阶段通常不要求读者手动修改全部参数，但应当让读者理解：导航行为效果与参数配置密切相关，不同机器人尺寸与传感器特性需要匹配不同参数组合，否则可能出现路径贴墙、绕障失败或震荡抖动等现象。

Navigation2 还支持与仿真环境深度结合。在 Gazebo 仿真中，可以同时运行机器人模型、传感器插件与导航系统，实现完整的“虚拟环境自主导航”实验流程。这种仿真导航方式在算法验证与课程实践中非常常见，能够在不依赖真实硬件的情况下完成导航系统全流程训练。



10.6.1 导航应用开发指南

在实际应用开发中，导航系统通常不会以默认参数直接投入使用，而需要根据机器人尺寸、传感器类型与运行环境进行参数调整。开发流程一般从仿真环境开始，在 Gazebo 中完成导航链路联调，再迁移到真实机器人平台进行实测验证。这样可以显著降低硬件调试风险。

应用开发时应重点关注坐标系配置是否正确、传感器数据是否稳定、代价地图参数是否与机器人尺寸匹配，以及控制器参数是否满足运动特性要求。调试过程中建议始终配合 rviz 可视化观察中间结果，通过可视化反馈判断系统行为是否符合预期。

10.7 本章实践 开发板实现

在循迹小车的基础升级，实现 ROS2 控制下的巡线小车实现

ROS2 的开发板安装

ROS2 功能包编写

小车功能实现
